

Curso: Go e System Design para processamento de arquivos em alto volume

Oct 7, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo projetar e implementar, em Go 1.27, um pipeline que recebe milhares de arquivos por dia, processa milhões de linhas e grava tudo no PostgreSQL sem estourar memória, sem duplicar dados e sem perder registros quando algo falha.

É a versão em Go do curso de .NET 10 sobre o mesmo tema: os mesmos dez módulos, o mesmo método **top-down** (o problema e os números, depois a arquitetura, depois cada componente, só então o código) e o mesmo projeto final. O system design é idêntico, porque ele não depende da linguagem; o que muda é a implementação de cada peça e, principalmente, o runtime: o GC do Go funciona de outro jeito, a concorrência é feita com goroutines e canais, e não há host genérico nem injeção de dependência. Cada módulo diz onde a solução em Go difere da de .NET e por quê.

Pré-requisitos

Sintaxe de Go (structs, interfaces, slices, goroutines, canais, tratamento de erros) e noções de SQL. O curso de APIs HTTP em Go cobre os idiomas da linguagem usados aqui (erros como valores, interfaces no consumidor, `context.Context`); não é pré-requisito, mas ajuda.

Exemplo condutor

Arquivos de retorno bancário em layout posicional (CNAB 240/400): linhas de largura fixa, chegada em lotes, processamento exatamente uma vez e tabelas grandes no destino. Tudo que você aprender vale para CSV, JSON Lines ou logs.

O worker deste curso consome as mesmas mensagens e grava nas mesmas tabelas que o worker em .NET. Num time real, os dois poderiam até conviver durante uma migração, processando partições diferentes da fila.

Stack usada

- Go 1.27 e a biblioteca padrão (`bufio`, `io`, `context`, `log/slog`, `uuid`, `runtime/trace`)
- PostgreSQL com `pgx` (incluindo o protocolo `COPY`) e `sqlc`

- AWS SDK para Go v2 (S3 e SQS) ou equivalentes
- `golang.org/x/sync/errgroup`
- OpenTelemetry para Go, `pprof` e benchmarks do pacote `testing`

Para quem vem do .NET

Conceito	Curso em .NET 10	Este curso em Go 1.27
Processo do worker	Generic Host + <code>BackgroundService</code>	<code>main</code> + <code>run</code> com <code>signal.NotifyContext</code>
Janela sobre memória	<code>Span<T></code> , <code>ReadOnlySpan<byte></code>	Fatias (<code>[]byte</code>) e subfatias
Leitura em streaming	<code>System.IO.Pipelines</code>	<code>bufio.Reader</code> / <code>bufio.Scanner</code>
Pools	<code>ArrayPool<T></code>	<code>sync.Pool</code>
GC	Geracional (gen0/1/2 + LOH)	Concorrente, não geracional, ajustado por <code>GOGC</code> e <code>GOMEMLIMIT</code>
Pipeline com back-pressure	<code>Channel<T></code> com limite	<code>chan T</code> com buffer
Grupo de tarefas	<code>Task.WhenAll</code> + cancelamento vinculado	<code>errgroup.WithContext</code>
Escrita em massa	<code>BeginBinaryImportAsync</code> (Npgsql)	<code>CopyFrom</code> (pgx)
Retry	Polly	Função própria de backoff com jitter
Benchmarks	BenchmarkDotNet	<code>testing.B</code> com <code>b.Loop()</code>

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	Quanto volume, com que prazo, com que garantias?	Uma ficha de requisitos com números
2. Arquitetura	Quais peças existem e como conversam?	O diagrama de referência e a tabela <code>arquivo</code>

Módulo	Pergunta que responde	O que você entrega
3. Runtime	O que o runtime do Go faz por mim e onde eu atrapalho?	Configuração de GC e memória, código sem alocação
4. I/O de arquivos	Como ler 2 GB sem carregar 2 GB?	Um leitor em streaming e um parser de bytes
5. Concorrência	Como paralelizar sem afogar o banco?	Um pipeline com canais e back-pressure
6. Escrita em massa	Como inserir 1 milhão de linhas em segundos?	Um escritor com <code>CopyFrom</code> e <code>upsert</code>
7. Leitura em massa	Como ler tabelas grandes sem travar nada?	Consultas em streaming e keyset
8. Confiabilidade	O que acontece quando algo quebra no meio?	Lease, retries, DLQ, outbox e reconciliação
9. Observabilidade	Como sei que está rápido e saudável?	Métricas, traces, perfis e benchmarks
10. Escala	Como crescer de 1 para N workers?	Deploy com autoscaling por fila
Projeto final	Tudo junto funciona?	O worker completo, testado sob carga

Sugestão de ritmo: um módulo por semana. Os Módulos 3, 4 e 5 são os que mais diferem do curso de .NET e merecem mais tempo.

Módulo 1 — O problema: anatomia de um pipeline de alto volume

Antes de escrever código, você precisa de três números: quantas linhas por segundo o sistema sustenta no pico, em quanto tempo um arquivo precisa estar processado e qual garantia de entrega o negócio exige. Sem eles, toda decisão de arquitetura é chute, em Go ou em qualquer linguagem.

1.1 Onde o jeito ingênuo quebra

O jeito ingênuo em Go é: `os.ReadFile`, `strings.Split(contenido, "\n")`, um `strconv.Atoi` por campo e um `INSERT` por linha. Ele quebra nos mesmos três pontos que em .NET:

1. **Memória:** `os.ReadFile` coloca o arquivo inteiro num `[]byte`, a conversão para `string` faz uma cópia completa e o `Split` cria um slice de strings. Um arquivo de 500 MB vira mais de 1 GB de heap antes da primeira linha ser processada.
2. **Banco:** um `INSERT` por linha é uma ida e volta de rede por linha. Com 1 ms de latência, 1 milhão de linhas custam mais de 16 minutos só de espera.
3. **Falha:** se o processo cai na linha 800 mil, não há como saber o que foi gravado. Reprocessar duplica; não reprocessar perde.

Em Go há um quarto ponto, específico da linguagem: **goroutines sem limite**. A tentação de escrever `for _, linha := range linhas { go gravar(linha) }` é grande, porque goroutines são baratas. O resultado são milhões de goroutines disputando um pool de 20 conexões, a memória explode e o banco recebe uma rajada que não aguenta. Goroutines são baratas; o que elas acessam, não.

1.2 Requisitos que definem a arquitetura

Requisito	Pergunta para o negócio	Por que muda o design
Vazão	Quantas linhas por segundo no pico?	Número de workers e técnica de escrita
Latência ponta a ponta	Em quanto tempo o dado precisa estar disponível?	Lote ou streaming contínuo
Garantia de entrega	Pode perder? Pode duplicar?	Idempotência, transações, checkpoint
Ordenação	A ordem importa?	Limita o paralelismo e exige particionamento
Tamanho dos arquivos	Qual o maior possível, não o médio?	Processar inteiro ou dividir em pedaços
Retenção e auditoria	Por quanto tempo guardar o original?	Armazenamento e rastreabilidade

Projete para o **pior arquivo** e a **hora de pico**. Arquivos bancários chegam concentrados em janelas curtas, e a média diária esconde isso.

1.3 Contas de guardanapo

O mesmo cenário hipotético do curso de .NET, para que os dois projetos sejam comparáveis:

- 5.000 arquivos por dia, média de 40.000 linhas, maior arquivo com 3 milhões de linhas

- 60% dos arquivos numa janela de 2 horas
- SLA de 15 minutos por arquivo

Total diário: 200 milhões de linhas. Na janela de pico, 120 milhões em 7.200 segundos: cerca de **16.700 linhas por segundo**, e com margem de 2×, um alvo de projeto de 35.000 linhas/s. O maior arquivo, sozinho, exige pelo menos 3.300 linhas/s para caber nos 15 minutos.

No banco, 200 milhões de linhas de 300 bytes são 60 GB por dia sem índices, mais de 20 TB por ano. Essa conta, no primeiro dia, já obriga a pensar em particionamento e retenção (Módulo 7).

1.4 Lei de Little: quantos escritores?

$$L = \lambda \cdot W$$

L é o número médio de itens em processamento ao mesmo tempo, λ a taxa de chegada e W o tempo que cada item passa no sistema. Com alvo de 35.000 linhas/s em lotes de 5.000 linhas, são 7 lotes por segundo; se cada lote leva 0,5 s para ser gravado, $L = 7 \times 0,5 = 3,5$, ou seja, pelo menos 4 escritores concorrentes. Se a escrita piora para 1 s, são 7. Em Go, L é literalmente o número de goroutines escritoras que você vai criar no Módulo 5, e também o número de conexões que elas vão ocupar.

1.5 Onde está o gargalo

- **I/O** (baixar do S3, ler disco): CPU ociosa esperando bytes. Remédio: buffers maiores, leitura e processamento sobrepostos.
- **CPU** (parsing, validação): núcleos em 100%. Remédio: código sem alocação (Módulos 3 e 4) e paralelismo até o número de núcleos.
- **Banco** (escrita, locks, índices): o worker espera. Remédio: lotes grandes, `COPY`, menos índices na staging e **concorrência limitada** (Módulos 5 e 6).

Em pipelines de arquivo o gargalo quase sempre é o banco. Todo o design gira em torno de protegê-lo: escrever em lotes, com concorrência limitada e com back-pressure, que é a capacidade de o estágio lento frear o rápido em vez de deixar a memória encher.

1.6 Garantias de entrega

- **At-most-once**: pode perder, nunca duplica. Inaceitável para dado financeiro.
- **At-least-once**: nunca perde, pode duplicar. É o que filas e retries entregam.
- **Exactly-once**: o que o negócio quer, obtido na prática como **at-least-once + idempotência**.

Essa equação volta no Módulo 8 e é a base de todo o design de confiabilidade.

Exercício do Módulo 1

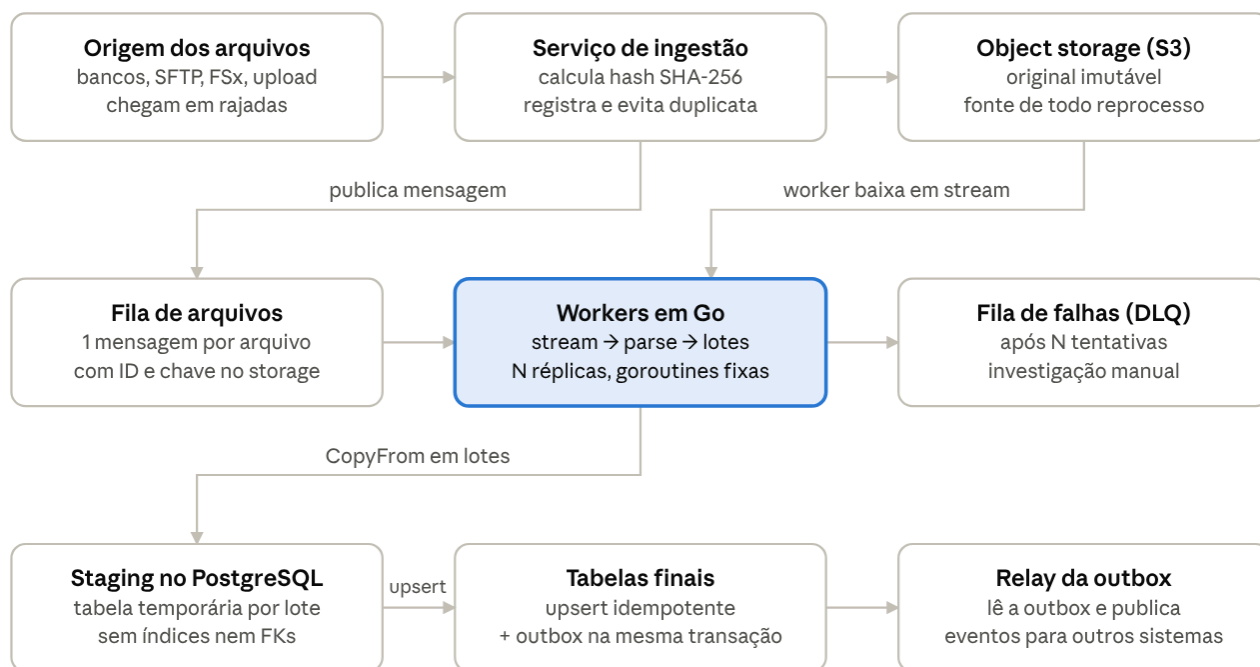
Preencha a ficha de requisitos do seu cenário (ou use o de exemplo): arquivos por dia, linhas por arquivo (média e máximo), janela de pico, SLA, garantia de entrega e retenção. Calcule linhas por segundo no pico, volume diário em GB e o número de escritores pela Lei de Little. Depois escreva a versão ingênua (`os.ReadFile` + `Split` + `INSERT` por linha), rode-a num arquivo de 200 mil linhas e anote tempo e pico de memória: esse é o ponto de partida que o resto do curso vai melhorar.

Módulo 2 — System design: a arquitetura de referência

A arquitetura é a mesma do curso de .NET, porque ela responde ao problema e não à linguagem: separar **receber** de **processar**, guardar o original intacto, distribuir o trabalho por uma fila, processar em workers sem estado que escrevem em lotes numa staging e consolidam nas tabelas finais com uma operação idempotente, e propagar eventos por uma outbox.

O arquivo entra uma vez, é processado em lotes e vira eventos pela outbox

Arquitetura de referência · o worker em Go em destaque é o foco dos Módulos 3 a 7



arquitetura de referência · 9 componentes, 3 camadas

Leia de cima para baixo: a primeira linha recebe e guarda, a segunda distribui, a terceira grava e propaga. Cada seta é um ponto de falha, e cada componente foi escolhido para que a falha seja recuperável.

2.1 Os seis princípios

1. **Separe ingestão de processamento.** Uma rajada de 2.000 arquivos é aceita em segundos e drenada no ritmo que o banco aguenta.
2. **O original é imutável e fica guardado.** Todo bug de parsing vira "corrigir e reprocessar", nunca "recuperar dado perdido".
3. **A unidade de trabalho é explícita:** o arquivo, ou um pedaço dele para arquivos gigantes. A mensagem aponta para a unidade, nunca carrega o conteúdo.
4. **Workers sem estado.** Um processo pode morrer a qualquer momento; tudo que importa (status, posse do arquivo) mora no banco.
5. **O banco é protegido:** lotes, concorrência limitada, back-pressure.
6. **Todo passo é idempotente**, o que torna retries seguros.

2.2 Os componentes

Ingestão. Recebe o arquivo (upload HTTP, como na API do curso de APIs em Go, ou leitura de SFTP/SMB), calcula o SHA-256 enquanto grava no storage, insere uma linha em `arquivo` com `UNIQUE (cliente_id, sha256)` e só então publica a mensagem. Duplicatas morrem na porta.

Object storage. Guarda o original com chave previsível (`retornos/2026/10/07/<id>`) e permite leitura em stream e por intervalo de bytes.

Fila. Uma mensagem por unidade de trabalho com o ID e a chave no storage. Dá distribuição entre workers, reentrega quando um worker some e uma métrica natural de atraso para o autoscaling.

Workers em Go. Leem o arquivo em stream com `bufio` , fazem parsing dos bytes sem criar strings, agrupam em lotes e gravam com `CopyFrom` . Internamente, um pipeline de goroutines ligadas por canais com buffer (Módulo 5).

DLQ. Mensagens que falham N vezes vão para uma fila separada, para que um arquivo envenenado não trave os outros.

Staging. Uma tabela temporária por lote, sem índices nem chaves estrangeiras, que recebe o `COPY` da forma mais rápida possível.

Tabelas finais. Recebem os dados por `INSERT ... SELECT ... ON CONFLICT` , com uma chave natural, e na mesma transação gravam os eventos na outbox.

Relay da outbox. Publica os eventos para outros sistemas, resolvendo o "gravei mas não publiquei" (Módulo 8).

2.3 O ciclo de vida de um arquivo e a tabela `arquivo`

Estado	Significado	Quem muda e para onde
RECEBIDO	Gravado no storage e registrado	Ingestão cria; worker muda para PROCESSANDO
PROCESSANDO	Um worker está com ele, com prazo (lease)	Worker muda para CONCLUIDO , REJEITADO ou FALHOU
CONCLUIDO	Todas as linhas consolidadas	Estado final
REJEITADO	Layout inválido; nada aplicado	Estado final; vai para análise
FALHOU	Erro técnico após N tentativas	Operação pode voltar para RECEBIDO

Toda transição é um `UPDATE` condicional, e o **lease** (`processando_ate`) permite que outro worker assuma se o dono morrer. A tabela é exatamente a do curso de .NET, compartilhada por todos os serviços do sistema:

```
CREATE TABLE arquivo (
  id                uuid          PRIMARY KEY,          -- UUID v7,
  gerado pela ingestão
  sequencia         bigint        GENERATED ALWAYS AS IDENTITY UNIQUE,
  cliente_id        uuid          NOT NULL,
  banco             text          NOT NULL,
  nome_original     text          NOT NULL,
  sha256            text          NOT NULL,
  chave_storage     text          NOT NULL,
  status            text          NOT NULL DEFAULT 'RECEBIDO'
  CHECK (status IN ('RECEBIDO', 'PROCESSANDO', 'CONCLUIDO',
    'REJEITADO', 'FALHOU')),
  tentativas        int           NOT NULL DEFAULT 0,
  worker_id         text          NULL,
  processando_ate   timestamptz  NULL,                -- lease
  linhas_processadas int         NOT NULL DEFAULT 0,
  linhas_rejeitadas int         NOT NULL DEFAULT 0,
  motivo           text          NULL,
  recebido_em       timestamptz  NOT NULL DEFAULT now(),
  data_recebimento  date         NOT NULL,
  CONSTRAINT uq_arquivo_conteudo UNIQUE (cliente_id, sha256)
);
```



```
CREATE INDEX ix_arquivo_cliente_sequencia ON arquivo (cliente_id, sequencia
DESC);
CREATE INDEX ix_arquivo_pendentes ON arquivo (recebido_em) WHERE status IN
('RECEBIDO', 'PROCESSANDO');
```

Em Go, os IDs v7 vêm do pacote `uuid` da biblioteca padrão (`uuid.NewV7()`, Go 1.27), ordenados por tempo para que as inserções caiam no fim do índice.

2.4 Arquivos gigantes: fan-out e fan-in

Um arquivo de 3 milhões de linhas num único worker fica limitado à vazão desse worker. A saída é dividi-lo:

1. Um passo de preparação calcula offsets de byte a cada N linhas. Em layout de largura fixa é aritmética: a linha k começa em $k \times (\text{tamanho da linha} + \text{quebra})$.
2. Ele cria registros em `arquivo_chunk` e publica uma mensagem por chunk.
3. Workers processam chunks em paralelo, lendo só o intervalo de bytes deles (leitura por `Range` no S3 ou `io.NewSectionReader` num arquivo local).
4. Quando o último chunk termina, o fan-in valida o trailer contra a soma dos chunks e marca o arquivo como `CONCLUIDO`.

2.5 Alternativas

Decisão	Opção A	Opção B	Escolha A quando	Escolha B quando
Distribuição	Fila gerenciada (SQS, Service Bus)	Tabela como fila (<code>FOR UPDATE SKIP LOCKED</code>)	Muitos consumidores, autoscaling	Volume moderado, menos infraestrutura
Chegada	Evento do storage	Polling de pasta	O storage emite eventos	Origem legada (SMB, SFTP)
Escrita	Staging + upsert	Upsert direto	Lotes grandes, muitos índices	Lotes pequenos, baixa latência
Divisão	Arquivo inteiro	Chunks	Até centenas de milhares de linhas	Milhões de linhas com SLA curto

A tabela como fila é especialmente natural em Go: uma goroutine faz `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1` em laço, e várias réplicas pegam itens diferentes sem se bloquear. Para alguns milhares de arquivos por dia, elimina uma peça de infraestrutura.

Exercício do Módulo 2

Desenhe a arquitetura para o seu cenário da ficha do Módulo 1. Para cada seta, escreva o que acontece se o destino ficar fora do ar por 10 minutos: o que se perde, o que se acumula e como o sistema se recupera. Se alguma resposta for "perde dados", o design ainda não está pronto. Depois crie a tabela `arquivo` num PostgreSQL local e escreva o `UPDATE` condicional que muda `RECEBIDO` para `PROCESSANDO`.

Módulo 3 — O runtime do Go: o que importa para vazão

O GC do Go é **concorrente e não geracional**: ele marca todo o heap vivo a cada ciclo, em paralelo com o programa, com pausas de parada total muito curtas. Não existe uma "geração jovem" barata como no .NET, então o custo do GC depende de duas coisas que você controla: **quanta memória fica viva e quantos bytes por segundo você aloca**. O objetivo do caminho quente (o código que roda por linha) é o mesmo do curso de .NET: zero alocação por linha. As ferramentas é que mudam.

3.1 Como o GC do Go decide quando trabalhar

Um ciclo de GC começa quando o heap atinge um **alvo** calculado a partir do heap vivo após o ciclo anterior:

```
alvo = heap\_vivo \times \left(1 + \frac{GOGC}{100}\right)
```

Com o padrão `GOGC=100`, se 200 MB sobreviveram ao último ciclo, o próximo começa quando o heap chegar a 400 MB. Duas consequências práticas para um worker de arquivos:

- **Alocar muito dispara ciclos com frequência.** Um parser que cria strings por campo gera centenas de MB por segundo de lixo, e o GC passa a rodar o tempo todo. Durante a marcação, o runtime reserva cerca de um quarto da CPU para o GC e ainda obriga as goroutines que alocam a "ajudar" (*mark assist*), o que aparece como lentidão no seu código.
- **Manter muito vivo encarece cada ciclo.** Sem gerações, cada ciclo percorre todo o heap vivo. Carregar o arquivo inteiro em memória não é só risco de OOM: é GC mais caro do início ao fim.

O GC do Go também não move objetos (não há compactação) e não tem um "Large Object Heap" separado. Buffers grandes não têm a penalidade especial do LOH do .NET, mas contam integralmente no heap vivo. Desde o Go 1.26, o coletor padrão é o Green Tea, que marca objetos pequenos com mais eficiência; você ganha isso sem configurar nada.

3.2 GOGC e GOMEMLIMIT

Variável	O que faz	Para um worker de arquivos
GOGC	Quanto o heap pode crescer além do vivo antes do próximo ciclo	Subir (por exemplo, 200) troca memória por menos ciclos, se houver folga
GOMEMLIMIT	Limite suave de memória total do runtime	Defina sempre, um pouco abaixo do limite do container (cerca de 90%)
GOMAXPROCS	Quantas threads executam Go ao mesmo tempo	Nada a fazer: desde o Go 1.25 segue o limite de CPU do container

O `GOMEMLIMIT` é o ajuste mais importante em Kubernetes. Sem ele, o runtime não sabe que existe um teto: com `GOGC=100` e 600 MB vivos, o alvo seria 1,2 GB, e um pod de 1 GiB seria morto por OOM antes do GC começar. Com `GOMEMLIMIT=900MiB`, o GC fica mais agressivo ao se aproximar do limite. Ele é "suave": se o heap vivo realmente não couber, o runtime limita o tempo de CPU gasto em GC em vez de entrar num ciclo infinito, e o processo fica lento antes de morrer, o que dá tempo para os alertas.

Os dois podem ser definidos por variável de ambiente ou no código (`debug.SetGCPercent`, `debug.SetMemoryLimit`), e o `runtime/metrics` expõe quanto tempo de CPU foi para o GC, que entra nas métricas do Módulo 9.

3.3 Escape analysis: pilha ou heap?

Em Go você não escolhe onde um valor vive; o compilador decide. Valores que comprovadamente não sobrevivem à função ficam na pilha (custo zero para o GC); os demais "escapam" para o heap. Para ver as decisões:

```
go build -gcflags='-m' ./internal/cnab 2>&1 | grep -E 'escapes|moved to heap'
```

As causas mais comuns de escape num caminho quente:

- **Interfaces e `any`**: passar um valor para um parâmetro `any` (como `fmt.Sprintf` ou `slog.Info`) normalmente faz ele escapar. Nunca formate strings por linha.
- **Ponteiros devolvidos** ou guardados em estruturas que vivem mais.
- **Closures capturadas por goroutines**.
- **`make` com tamanho que só se sabe em tempo de execução**, embora o Go 1.26 tenha passado a alocar mais desses casos na pilha.

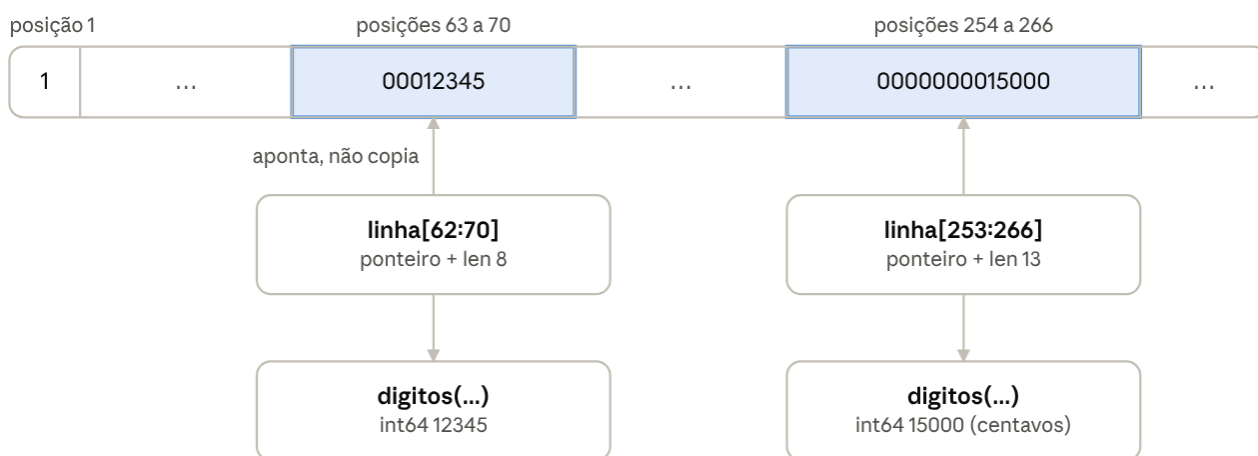
O que não escapa, nesse código: structs pequenas passadas e devolvidas por valor (como o `RegistroRetorno` abaixo), erros sentinela pré-alocados e subfatias de um buffer existente.

3.4 Fatias de bytes: o Span do Go

Uma fatia (`[]byte`) é um cabeçalho de três palavras: ponteiro, tamanho e capacidade. Fatiar (`linha[62:70]`) cria um novo cabeçalho apontando para os mesmos bytes, sem copiar nada. É o que o `ReadOnlySpan<byte>` faz no .NET, com uma diferença importante: em Go a fatia pode ser guardada em qualquer lugar (não há a restrição de `ref struct`), e isso traz duas armadilhas.

Uma subfatia é uma janela sobre o mesmo array: fatiar não copia nada

Uma linha CNAB de 400 bytes dentro do buffer do `bufio` · fora de escala



Guardar uma subfatia mantém vivo o buffer inteiro, que o `bufio` vai reescrever: copie com `bytes.Clone`

subfatias de uma linha CNAB · 2 janelas, nenhuma cópia

Armadilha 1: o buffer é reaproveitado. O `bufio.Scanner` devolve em `Bytes()` uma fatia do próprio buffer interno, válida só até o próximo `Scan()`. Se você guardar essa fatia num lote para gravar depois, quando for gravar ela já conterá os bytes de outra linha. Parseie para valores (números, datas) imediatamente; se precisar guardar bytes, copie com `bytes.Clone`.

Armadilha 2: uma subfatia mantém o array inteiro vivo. Guardar `buffer[10:20]` de um buffer de 1 MB impede o GC de liberar o MB inteiro. Para guardar um pedaço pequeno de algo grande, copie-o.

E a conversão `string(b)` aloca e copia, exceto em casos que o compilador otimiza (comparações, `switch string(b)` e busca em mapa com `m[string(b)]`). Para campos numéricos, a solução é não converter: parsear os bytes direto, como abaixo. Para campos

de texto que você realmente guarda, como o nome do pagador, decodifique só aquele campo (arquivos bancários costumam vir em ISO-8859-1; o pacote `golang.org/x/text/encoding/charmap` tem o decodificador).

3.5 O parser sem alocação

O equivalente ao `Cnab400Parser` do curso de .NET (posições ilustrativas; cada banco tem as suas):

```
package cnab

const TamanhoLinha = 400

type RegistroRetorno struct {
    NossoNumero      int64
    ValorPagoCentavos int64
    DataOcorrencia   time.Time
    NumeroLinha      int32
    CodigoOcorrencia uint8
}

// Erros sentinela: criados uma vez, devolvidos sem alocar
var (
    ErrTamanho      = errors.New("tamanho de linha inválido")
    ErrNaoDetalhe   = errors.New("registro não é de detalhe")
    ErrNossoNumero  = errors.New("nosso número inválido")
    ErrOcorrencia   = errors.New("código de ocorrência inválido")
    ErrData         = errors.New("data inválida")
    ErrValor        = errors.New("valor pago inválido")
    ErrTrailer      = errors.New("trailer inválido")
)

func ParseDetalhe(linha []byte, numeroLinha int) (RegistroRetorno, error) {
    if len(linha) != TamanhoLinha {
        return RegistroRetorno{}, ErrTamanho
    }
    if linha[0] != '1' {
        return RegistroRetorno{}, ErrNaoDetalhe
    }
    // Posições do layout são 1-based e inclusivas: 63 a 70 vira linha[62:70]
    nosso, ok := digitos(linha[62:70])
    if !ok {
        return RegistroRetorno{}, ErrNossoNumero
    }
    ocorrencia, ok := digitos(linha[108:110])
    if !ok {
```

```

        return RegistroRetorno{}, ErrOcorrencia
    }
    data, ok := dataDDMMAA(linha[110:116])
    if !ok {
        return RegistroRetorno{}, ErrData
    }
    valor, ok := digitos(linha[253:266])
    if !ok {
        return RegistroRetorno{}, ErrValor
    }
    return RegistroRetorno{
        NossoNumero: nosso, ValorPagoCentavos: valor, DataOcorrencia: data,
        NumeroLinha: int32(numeroLinha), CodigoOcorrencia: uint8(ocorrencia),
    }, nil
}

// ParseTrailer lê quantidade de registros e valor total, usados na
reconciliação (Módulo 8)
func ParseTrailer(linha []byte) (quantidade int, totalCentavos int64, err
error) {
    if len(linha) != TamanhoLinha || linha[0] != '9' {
        return 0, 0, ErrTrailer
    }
    q, ok1 := digitos(linha[17:25])
    t, ok2 := digitos(linha[25:39])
    if !ok1 || !ok2 {
        return 0, 0, ErrTrailer
    }
    return int(q), t, nil
}

func digitos(campo []byte) (int64, bool) {
    var v int64
    for _, b := range campo {
        d := b - '0' // em byte, valores abaixo de '0' dão a volta e ficam
maiores que 9
        if d > 9 {
            return 0, false
        }
        v = v*10 + int64(d)
    }
    return v, true
}

func dataDDMMAA(c []byte) (time.Time, bool) {
    dd, ok1 := digitos(c[0:2])

```

```

    mm, ok2 := digitos(c[2:4])
    aa, ok3 := digitos(c[4:6])
    if !ok1 || !ok2 || !ok3 || mm < 1 || mm > 12 || dd < 1 {
        return time.Time{}, false
    }
    t := time.Date(2000+int(aa), time.Month(mm), int(dd), 0, 0, 0, 0,
time.UTC)
    if t.Day() != int(dd) { // 31/02 é normalizado para março: a data era
inválida
        return time.Time{}, false
    }
    return t, true
}

```

O que observar:

- **Struct por valor.** `RegistroRetorno` é devolvido por valor e fica na pilha ou dentro do slice do lote, nunca como objeto solto no heap. A ordem dos campos (os de 8 bytes primeiro) evita bytes de preenchimento.
- **Erros sentinela.** `ErrValor` é criado uma vez, então devolver erro não aloca. Quem chama compara com `errors.Is` e acrescenta o número da linha só no relatório, não no caminho quente.
- **`time.Date` não aloca** com `time.UTC`, e o truque de comparar `t.Day()` valida a data sem uma tabela de dias por mês.
- **O `b - '0'` em byte** faz a validação com uma comparação só, o mesmo truque do `(uint)(b - '0') > 9` do C#.

3.6 sync.Pool: reutilize em vez de alocar

O equivalente ao `ArrayPool<T>` é o `sync.Pool`, um cache de objetos temporários por processador. No worker, o uso certo é reaproveitar os slices de lote entre o agrupador e os escritores (Módulo 5):

```

const TamanhoLote = 5000

var poolDeLotes = sync.Pool{
    New: func() any {
        s := make([]cnab.RegistroRetorno, 0, TamanhoLote)
        return &s
    },
}

func obterLote() *[]cnab.RegistroRetorno { return poolDeLotes.Get().(*[]cnab.RegistroRetorno) }

func devolverLote(l *[]cnab.RegistroRetorno) {
    *l = (*l)[:0] // zera o tamanho, mantém a capacidade
    poolDeLotes.Put(l)
}

```

Três regras: guarde **ponteiros** para slices no pool (guardar o slice direto alocaria a cada `Put`, porque ele seria convertido para `any`); nunca use o objeto depois de devolvê-lo; e não conte com o pool para guardar nada, porque o runtime pode esvaziá-lo a qualquer ciclo de GC. Ele é um cache, não um estoque.

3.7 O processo do worker

Go não tem Generic Host. O equivalente é o padrão `main` + `run` (o mesmo do curso de APIs em Go): o `main` cria um contexto cancelado no `SIGTERM` e chama `run`, que monta tudo, roda o laço de consumo e devolve erro se algo fatal acontecer.

```

func main() {
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
    syscall.SIGTERM)
    defer stop()
    if err := run(ctx, os.Getenv); err != nil {
        slog.Error("worker encerrado com erro", "erro", err)
        os.Exit(1) // o Kubernetes reinicia o pod
    }
}

```

O que o host do .NET fazia por convenção, aqui é explícito: a configuração é validada no início do `run` (o `ValidateOnStart`), um erro fatal no laço devolve erro e encerra o processo (o `BackgroundServiceExceptionBehavior.StopHost`) e o cancelamento do contexto no `SIGTERM` é o início do desligamento gracioso (Módulo 8.8). Não há nada parecido com "não faça trabalho síncrono antes do primeiro `await`": o `run` é código comum, de cima para baixo.

Exercício do Módulo 3

Escreva dois benchmarks com `b.Loop()` sobre 100 mil linhas: um com `string(linha[62:70]) + strconv.ParseInt` por campo, outro com `ParseDetalle`. Rode com `go test -bench . -benchmem` e compare `ns/op`, `B/op` e `allocs/op`; a versão de bytes deve ter zero alocações. Depois rode o worker ingênuo do Módulo 1 com `GODEBUG=gctrace=1` e conte quantos ciclos de GC acontecem; repita com o parser novo.

Módulo 4 — I/O de arquivos de alta performance

O objetivo é o mesmo do curso de .NET: ler um arquivo de qualquer tamanho com **memória constante**. Em Go, a peça central é o pacote `bufio`, que lê em blocos e entrega cada linha como uma fatia do próprio buffer, sem cópia. Ele cumpre o papel do `System.IO.Pipelines` com uma API bem mais simples, porque a linguagem não tem a separação entre código síncrono e `async`: toda leitura é bloqueante do ponto de vista da goroutine e não bloqueia nenhuma thread do sistema.

4.1 Os três níveis de leitura

Abordagem	Memória	Alocação por linha	Quando usar
<code>os.ReadFile</code> + <code>bytes.Split</code>	Proporcional ao arquivo	Um cabeçalho de fatia por linha, todos vivos	Arquivos de configuração, nunca dados
<code>bufio.Scanner</code> com <code>Text()</code>	Constante	Uma <code>string</code> por linha	Volume moderado, protótipos
<code>bufio.Scanner</code> com <code>Bytes()</code>	Constante	Zero	Alto volume, caminho quente

A diferença entre as duas últimas linhas é uma chamada: `Text()` copia a linha para uma string nova, `Bytes()` devolve a fatia do buffer interno (válida só até o próximo `Scan`, como visto no Módulo 3.4). O `Scanner` com a função padrão de linhas já remove o `\r` do fim (arquivos com quebra Windows), devolve a última linha mesmo sem quebra no final e resolve sozinho a linha que começa num bloco e termina no seguinte, o problema que o `PipeReader` resolvia no .NET.

Um limite importante: por padrão, o `Scanner` recusa linhas maiores que 64 KiB e para com `bufio.ErrTooLong`. Para CNAB isso nunca acontece, mas um arquivo corrompido sem quebras de linha produziria exatamente esse erro. Configure o buffer explicitamente e sempre verifique `sc.Err()` no fim.

4.2 Abrindo arquivos e lendo pedaços

Para arquivos locais (ou num volume montado, como FSx via SMB), `os.Open` e um `bufio` com buffer generoso bastam. Para ler só um intervalo (um chunk do fan-out do Módulo 2.4), use `io.NewSectionReader`, que lê por offset com `ReadAt` e é seguro para várias goroutines lerem o mesmo arquivo ao mesmo tempo:

```
f, err := os.Open(caminho)
if err != nil {
    return err
}
defer f.Close()

chunk := io.NewSectionReader(f, inicio, fim-inicio) // só os bytes deste
chunk
err = leitor.Ler(ctx, chunk, arquivoID, lotes, contexto)
```

4.3 O leitor: linha, parse, lote

No curso de .NET, o leitor entregava registros um a um num canal, e um estágio separado os agrupava em lotes. Em Go, o leitor pode montar os lotes ele mesmo, com slices do `sync.Pool` do Módulo 3.6, e mandar só lotes prontos ao canal. É um estágio a menos, uma operação de canal a cada 5.000 linhas em vez de uma por linha, e o design fica mais simples de ler:

```
package leitor

// O que o leitor informa sobre o arquivo, além dos lotes (implementado no
Módulo 8)
type ContextoDeLeitura interface {
    RegistrarErro(numeroLinha int, motivo error)
    RegistrarTrailer(quantidade int, totalCentavos int64)
    RegistrarFim(linhasLidas int)
}

type Lote struct {
    ArquivoID uuid.UUID
    Numero    int
    Itens     *[]cnab.RegistroRetorno // vem do sync.Pool; quem recebe
devolve
}

var ErrLayoutInvalido = errors.New("layout de arquivo inválido")

func Ler(ctx context.Context, origem io.Reader, arquivoID uuid.UUID,
```

```

saida chan<- Lote, c ContextoDeLeitura) error {

    sc := bufio.NewScanner(origem)
    sc.Buffer(make([]byte, 256<<10), 1<<20) // buffer de 256 KiB; linha
máxima de 1 MiB

    lote := obterLote()
    defer func() { devolverLote(lote) }() // o lote que estiver em mãos no
fim volta ao pool
    numero := 0

    enviar := func() error {
        numero++
        select {
            case saida <- Lote{ArquivoID: arquivoID, Numero: numero, Itens:
lote}: // cheio = espera
                lote = obterLote() // o anterior agora pertence ao escritor
                return nil
            case <-ctx.Done():
                return ctx.Err()
        }
    }

    n := 0
    for sc.Scan() {
        n++
        linha := sc.Bytes()
        if len(linha) == 0 {
            continue
        }
        switch linha[0] {
            case '0': // header
                if err := validarHeader(linha); err != nil {
                    return fmt.Errorf("%w: %w", ErrLayoutInvalido, err)
                }
                continue
            case '9': // trailer: totais para a reconciliação
                q, total, err := cnab.ParseTrailer(linha)
                if err != nil {
                    c.RegistrarErro(n, err)
                } else {
                    c.RegistrarTrailer(q, total)
                }
                continue
        }
    }
}

```

```

    reg, err := cnab.ParseDetalhe(linha, n)
    if err != nil {
        c.RegistrarErro(n, err) // linha inválida não interrompe a
leitura
        continue
    }
    *lote = append(*lote, reg) // cópia do valor: não guarda nada do
buffer do Scanner
    if len(*lote) == cap(*lote) {
        if err := enviar(); err != nil {
            return err
        }
    }
}
if err := sc.Err(); err != nil {
    return fmt.Errorf("leitura na linha %d: %w", n+1, err)
}
if len(*lote) > 0 {
    if err := enviar(); err != nil {
        return err
    }
}
c.RegistrarFim(n)
return nil
}

```

Pontos didáticos, um por um:

1. **O `select` no envio é o back-pressure.** Se o canal de lotes está cheio porque os escritores estão lentos, o leitor espera ali, e a leitura do arquivo (ou do S3) para junto. Se o contexto for cancelado enquanto espera, ele sai em vez de ficar preso para sempre, que é o vazamento de goroutine clássico.
2. **Posse explícita do lote.** Depois do envio, o leitor pega um lote novo e nunca mais toca no antigo. O `defer` devolve ao pool o lote que estiver em mãos quando a função terminar, seja por sucesso, erro ou cancelamento.
3. **`append` copia o valor.** `RegistroRetorno` só tem números e uma data; nenhum campo aponta para o buffer do `Scanner`, então o lote continua válido depois que o buffer é reescrito. Se um dia o registro ganhar um campo `[]byte`, ele precisa ser copiado.
4. **Erros de linha são registrados, erros de arquivo interrompem.** Uma linha de detalhe inválida vai para o contexto e a leitura segue; um header inválido devolve `ErrLayoutInvalido`, que o worker trata como rejeição sem retry (Módulo 8). O `%w: %w` (dois erros embrulhados, desde o Go 1.20) mantém os dois reconhecíveis por `errors.Is`.

4.4 Lendo direto do S3

O `Body` de um `GetObject` do SDK v2 é um `io.ReadCloser` em streaming; passe-o direto para `Ler`, sem `io.ReadAll`:

```
entrada := &s3.GetObjectInput{Bucket: aws.String(bucket), Key:
aws.String(chave)}
if chunk != nil {
    entrada.Range = aws.String(fmt.Sprintf("bytes=%d-%d", chunk.Inicio,
chunk.Fim-1)) // só o intervalo do chunk
}
obj, err := s.cliente.GetObject(ctx, entrada)
if err != nil {
    return fmt.Errorf("baixar %s: %w", chave, err)
}
defer obj.Body.Close()

return leitor.Ler(ctx, obj.Body, arquivoID, lotes, contexto)
```

Os mesmos dois cuidados do curso de .NET: uma conexão que cai no meio devolve erro de leitura, e o arquivo é reprocessado inteiro (seguro porque a escrita é idempotente); e se o processamento for lento a ponto de a conexão HTTP expirar enquanto o leitor espera no canal, baixe primeiro para um arquivo temporário (`os.CreateTemp` + `io.Copy`) e leia de lá.

4.5 Hash enquanto copia

Na ingestão, o SHA-256 sai do mesmo passo que grava o arquivo, com uma linha:

```
h := sha256.New()
if _, err := io.Copy(io.MultiWriter(destino, h), origem); err != nil {
    return "", err
}
return hex.EncodeToString(h.Sum(nil)), nil
```

`io.MultiWriter` escreve cada bloco nos dois destinos, e o `io.Copy` usa um buffer de 32 KiB reaproveitado durante toda a cópia. É a versão Go do laço com `IncrementalHash` do curso de .NET, e também o mesmo truque do `io.TeeReader` usado no upload da API em Go.

4.6 Escrevendo arquivos

Para gerar arquivos de remessa ou relatórios, formate direto em bytes num slice reaproveitado e escreva por um `bufio.Writer`:

```
// anexa v com exatamente n dígitos, zeros à esquerda (campos numéricos do
CNAB)
func anexarNumero(dst []byte, v int64, n int) []byte {
    var tmp [20]byte
    s := strconv.AppendInt(tmp[:0], v, 10) // formata na pilha, sem alocar
    for i := len(s); i < n; i++ {
        dst = append(dst, '0')
    }
    return append(dst, s...)
}

w := bufio.NewWriterSize(arquivo, 256<<10)
linha := make([]byte, 0, 402)
for _, r := range registros {
    linha = linha[:0]
    linha = append(linha, '1')
    linha = anexarNumero(linha, r.ValorPagoCentavos, 13)
    // ... demais campos
    linha = append(linha, '\r', '\n')
    if _, err := w.Write(linha); err != nil {
        return err
    }
}
if err := w.Flush(); err != nil { // sem Flush, o fim do arquivo fica no
buffer
    return err
}
```

Escreva sempre num arquivo temporário e renomeie no fim com `os.Rename`, que é atômico no mesmo sistema de arquivos: quem consome nunca vê um arquivo pela metade. No S3, use o `manager.Uploader` do SDK, que envia partes em paralelo e só publica o objeto quando todas chegam.

4.7 Arquivos compactados

`gzip.NewReader(origem)` funciona em streaming: embrulhe o reader e passe adiante. Arquivos `.zip` são diferentes, porque o índice fica no fim: `zip.NewReader` exige um `io.ReaderAt` e o tamanho total. Para zip vindo do S3, baixe para um arquivo temporário e abra com `zip.OpenReader`.

Exercício do Módulo 4

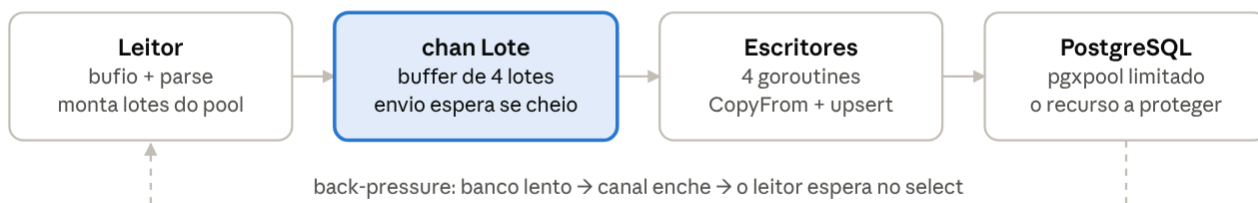
Gere um arquivo sintético de 2 milhões de linhas de 400 posições e leia-o três vezes: com `os.ReadFile` + `bytes.Split`, com `Scanner.Text()` e com o `leitor.Ler` (com um consumidor que só devolve os lotes ao pool). Meça tempo total e pico de memória (`/usr/bin/time -v` ou o perfil de heap) e conte os ciclos de GC com `GODEBUG=gctrace=1`. Depois corrompa o arquivo removendo todas as quebras de linha e confirme que o leitor devolve `bufio.ErrTooLong` em vez de consumir memória sem limite.

Módulo 5 — Concorrência e back-pressure

Dentro do worker, o processamento de um arquivo é um leitor e alguns escritores ligados por um **canal com buffer**. O buffer é o limite: quando o banco desacelera, o canal enche, o `select` de envio do leitor espera, e a leitura do arquivo para junto. Todas as goroutines rodam num `errgroup`, de modo que, se uma falha, o contexto é cancelado e todas as outras param.

Um canal com buffer faz o banco ditar o ritmo da leitura

Pipeline interno do worker em Go · todas as goroutines no mesmo `errgroup`



pipeline do worker · 1 leitor, 1 canal, 4 escritores

Comparado ao curso de .NET, há um estágio a menos (o leitor já monta os lotes, Módulo 4.3) e nenhum código de propagação de erro entre canais: o `errgroup` e o `close` do canal fazem esse papel.

5.1 Goroutines, o escalonador e o que não é preciso fazer

O runtime do Go escalona goroutines sobre `GOMAXPROCS` threads do sistema. Quando uma goroutine espera rede (o banco, o S3), ela é estacionada e a thread pega outra goroutine; quando espera uma chamada de sistema bloqueante (leitura de arquivo local), o runtime cria outra thread para não parar as demais. Disso saem duas diferenças grandes em relação ao .NET:

- Não há `async/await`, e não há o problema de *ThreadPool starvation* por código bloqueante. Código que espera I/O é escrito de forma sequencial e simples.

- **O limite não vem de threads, vem de você.** Criar uma goroutine custa alguns KB de pilha; criar um milhão é possível e quase sempre é um erro. Toda concorrência num worker é explicitamente limitada: um número fixo de escritores, um número fixo de arquivos em paralelo.

Trabalho de CPU (parsing) escala até o número de núcleos; trabalho de banco escala até o ponto de saturação do banco, que é bem antes.

5.2 Canais: as regras que importam

Um canal com buffer (`make(chan Lote, 4)`) aceita envios sem esperar até encher; depois disso, cada envio espera um recebimento. Em Go **não existe canal sem limite**, o que é ótimo para pipelines: o back-pressure vem de graça. As regras:

1. **Quem envia fecha.** `close(ch)` sinaliza "não há mais nada", e um `for lote := range ch` no receptor termina sozinho. Fechar do lado receptor, ou fechar duas vezes, causa `panic`; enviar num canal fechado também.
2. **Todo envio ou recebimento que pode esperar tem uma saída por cancelamento:** `select { case ch <- v: ...; case <-ctx.Done(): return ctx.Err() }`. Uma goroutine parada para sempre num envio é o vazamento de goroutine mais comum, e o perfil `goroutineleak` do Go 1.27 (Módulo 9) existe para encontrá-lo.
3. **O que passa pelo canal muda de dono.** Depois de enviar um lote, o leitor não toca mais nele; o escritor que o recebeu o devolve ao pool. Um ponteiro compartilhado por duas goroutines sem essa disciplina é uma corrida de dados, que só o `go test -race` encontra com confiabilidade.

5.3 O orquestrador do pipeline

```
package pipeline

type Escritor interface {
    Gravar(ctx context.Context, lote leitor.Lote) error
}

type Opcoes struct {
    CapacidadeLotes int // buffer do canal (4)
    Escritores      int // goroutines escritoras (4)
}

func ProcessarArquivo(ctx context.Context, arquivoID uuid.UUID, origem
io.Reader,
    c leitor.ContextoDeLeitura, escritor Escritor, o Opcoes) error {

    g, ctx := errgroup.WithContext(ctx) // ctx é cancelado no primeiro erro
```



```

de qualquer goroutine
    lotes := make(chan leitor.Lote, o.CapacidadeLotes)

    g.Go(func() error {
        defer close(lotes) // sem mais lotes: os escritores saem do range
        return leitor.Ler(ctx, origem, arquivoID, lotes, c)
    })

    for range o.Escritores {
        g.Go(func() error {
            for lote := range lotes {
                err := escritor.Gravar(ctx, lote)
                devolverLote(lote.Itens) // gravado ou não, o slice volta ao
pool
                if err != nil {
                    return err
                }
            }
            return nil
        })
    }

    return g.Wait() // espera todas e devolve o primeiro erro
}

```

Compare com as cerca de 100 linhas do orquestrador em .NET. O que observar:

- **Propagação de erro.** Se um escritor falha, o `errgroup` cancela o `ctx`; o leitor, se estiver esperando no `select` de envio, sai com `ctx.Err()`; o `defer close(lotes)` fecha o canal; os outros escritores terminam o `range` (ou falham rápido no `Gravar` com contexto cancelado). `g.Wait()` devolve o **primeiro** erro, que é a causa real, não o "contexto cancelado" que veio depois.
- **Desligamento.** O `ctx` de entrada vem do `signal.NotifyContext` do `main`: um `SIGTERM` cancela tudo do mesmo jeito que um erro.
- **Lotes perdidos no cancelamento.** Lotes que ficarem no buffer do canal quando tudo para são coletados pelo GC; o pool só deixa de reaproveitá-los. Não há vazamento.
- **for range o.Escritores** percorre um inteiro (Go 1.22), e cada goroutine captura suas próprias variáveis de laço.

5.4 Vários arquivos ao mesmo tempo

Arquivos diferentes são independentes: a falha de um não deve cancelar os outros. Por isso o laço de consumo não usa `errgroup.WithContext`, e sim um semáforo (um canal com buffer usado como contador) e um `sync.WaitGroup`:

```

func (w *Worker) Consumir(ctx context.Context) {
    sem := make(chan struct{}, w.Paralelo) // 2
    var wg sync.WaitGroup
    defer wg.Wait() // no desligamento, espera os arquivos em andamento

    for msg := range w.Fila.Mensagens(ctx) { // o canal fecha quando ctx é
cancelado
        select {
        case sem <- struct{}{}:
        case <-ctx.Done():
            return
        }
        wg.Go(func() { // Go 1.25: Add + go + Done numa chamada
            defer func() { <-sem }()
            w.processarMensagem(ctx, msg) // trata os próprios erros (Módulo
8)
        })
    }
}

```

Cuidado com a multiplicação: 2 arquivos × 4 escritores = 8 conexões por réplica, mais as de lease e metadados. É esse número que define o `MaxConns` do `pgxpool` e, multiplicado pelas réplicas, o orçamento de conexões do banco (Módulo 10).

5.5 Dimensionando

A concorrência certa é empírica, e o método é o do curso de .NET: uma estimativa pela Lei de Little, um teste de carga com 1, 2, 4, 8 e 16 escritores totais e a escolha do menor número que entrega cerca de 90% da vazão máxima. Acima do ponto de saturação, mais escritores só disputam locks e I/O do banco e a vazão cai. Com o pool do `pgx` configurado com `MaxConns` igual ao total de escritores mais uma margem, um escritor a mais do que o pool comporta não dá erro: ele espera uma conexão livre, e essa espera aparece na métrica de aquisição do pool (Módulo 9).

5.6 Quando a ordem importa

Com vários escritores, lotes são gravados fora de ordem. Se eventos do mesmo título precisam ser aplicados em ordem, há duas saídas, como no .NET:

- **Particionar por chave:** em vez de um canal, N canais, e cada registro vai para `canais[nossoNumero % N]`, com um escritor por canal. Todos os eventos de uma chave passam pelo mesmo escritor, em ordem.
- **Resolver no banco:** gravar tudo com o número da linha e consolidar em SQL ordenando por chave e linha. Costuma ser a opção mais simples e mais robusta.

5.7 Armadilhas de concorrência em Go

- **Goroutine sem saída:** envio ou recebimento sem `case <-ctx.Done()`.
- **Goroutines sem limite:** `go f()` dentro de um laço sobre dados de entrada.
- **Fechar canal do lado errado**, fechar duas vezes ou enviar depois de fechar: `panic`.
- **Compartilhar um `pgx.Conn`** entre goroutines: uma conexão não é segura para uso concorrente; o `pgxpool.Pool` é.
- **Esquecer o `Wait`**: a função retorna com goroutines ainda rodando, que tocam em dados que o chamador já considera livres.
- **Mexer num lote depois de enviá-lo:** corrida de dados silenciosa. Rode todo teste com `-race`.

Exercício do Módulo 5

Monte o `ProcessarArquivo` com um `Escritor` falso que dorme 50 ms por lote. Processe o arquivo do Módulo 4 e confirme pelo perfil de heap que a memória fica estável mesmo com o escritor lento. Depois faça o escritor devolver erro no lote 10 e confirme que o leitor para logo em seguida e que `ProcessarArquivo` devolve aquele erro, não `context.Canceled`. Por fim, remova o `case <-ctx.Done()` do envio no leitor, repita o teste do erro e encontre a goroutine presa em `/debug/pprof/goroutineleak`.

Módulo 6 — Escrita em massa no banco

A receita é a mesma do curso de .NET, porque ela é do PostgreSQL e não da linguagem: para cada lote, uma transação; dentro dela, o lote vai para uma tabela temporária pelo protocolo `COPY` e é consolidado na tabela final por um único `INSERT ... SELECT ... ON CONFLICT`, que também grava os eventos da outbox. Em Go, o `COPY` é o método `CopyFrom` do `pgx`.

6.1 As técnicas, da pior para a melhor

Uma linha por `INSERT` paga, a cada linha, ida e volta de rede, análise do comando, commit (com `fsync`) e manutenção de índices. As alternativas em Go:

Técnica	Idas ao banco por lote de 5.000	Observações
<code>Exec</code> por linha	5.000 (+ 5.000 commits sem transação)	Inaceitável em volume

Técnica	Idas ao banco por lote de 5.000	Observações
<code>pgx.Batch</code> com um <code>INSERT</code> por linha	1 (os comandos vão juntos)	Bom para lotes pequenos e comandos variados; o servidor ainda executa 5.000 inserções
<code>INSERT ... VALUES (...), (...)</code>	1 a poucas	Limite de 65.535 parâmetros; SQL gigante
<code>INSERT ... SELECT FROM unnest(\$1::bigint[], ...)</code>	1	Um parâmetro por coluna; o <code>pgx</code> converte slices Go em arrays
<code>CopyFrom</code>	1 (stream)	O mais rápido; precisa de staging para fazer upsert

6.2 O modelo de dados

As tabelas são as mesmas do curso de .NET, compartilhadas pelos workers das duas linguagens:

```
CREATE TABLE retorno_detalhe (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  arquivo_id  uuid      NOT NULL REFERENCES arquivo (id),
  numero_linha int      NOT NULL,
  nosso_numero bigint   NOT NULL,
  codigo_ocorrendia smallint NOT NULL,
  data_ocorrendia date   NOT NULL,
  valor_pago_centavos bigint NOT NULL,
  gravado_em  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT uq_retorno_linha UNIQUE (arquivo_id, numero_linha)
);
CREATE INDEX ix_retorno_nosso_numero ON retorno_detalhe (nosso_numero);

CREATE TABLE outbox (
  id          uuid PRIMARY KEY,
  tipo        text      NOT NULL,
  chave       text      NOT NULL,
  payload     jsonb      NOT NULL,
  criado_em   timestamptz NOT NULL DEFAULT now(),
  publicado_em timestamptz NULL
);
CREATE INDEX ix_outbox_pendentes ON outbox (criado_em) WHERE publicado_em IS NULL;
```

A chave natural (`arquivo_id`, `numero_linha`) é o que torna a gravação idempotente: reprocessar um arquivo produz exatamente os mesmos pares.

6.3 O escritor completo

```
package escrita

const criarTemporaria = `
CREATE TEMP TABLE IF NOT EXISTS tmp_retorno (
    arquivo_id uuid, numero_linha int, nosso_numero bigint,
    codigo_ocorrencia smallint, data_ocorrencia date, valor_pago_centavos
    bigint
) ON COMMIT DELETE ROWS`

var colunas = []string{"arquivo_id", "numero_linha", "nosso_numero",
    "codigo_ocorrencia", "data_ocorrencia", "valor_pago_centavos"}

const consolidar = `
WITH inseridos AS (
    INSERT INTO retorno_detalhe AS d
        (arquivo_id, numero_linha, nosso_numero, codigo_ocorrencia,
        data_ocorrencia, valor_pago_centavos)
    SELECT arquivo_id, numero_linha, nosso_numero, codigo_ocorrencia,
        data_ocorrencia, valor_pago_centavos
    FROM tmp_retorno
    ON CONFLICT (arquivo_id, numero_linha) DO NOTHING
    RETURNING d.id, d.nosso_numero, d.codigo_ocorrencia,
    d.valor_pago_centavos
)
INSERT INTO outbox (id, tipo, chave, payload)
SELECT gen_random_uuid(), 'retorno.ocorrencia', i.nosso_numero::text,
    jsonb_build_object('detalheId', i.id, 'nossoNumero', i.nosso_numero,
        'ocorrencia', i.codigo_ocorrencia,
        'valorPagoCentavos', i.valor_pago_centavos)
FROM inseridos i`

type EscritorPostgres struct {
    pool *pgxpool.Pool
}

func (e *EscritorPostgres) Gravar(ctx context.Context, lote leitor.Lote)
error {
    itens := *lote.Itens
    return pgx.BeginFunc(ctx, e.pool, func(tx pgx.Tx) error {
        if _, err := tx.Exec(ctx, criarTemporaria); err != nil {
            return fmt.Errorf("tabela temporária: %w", err)
        }
    })
}
```

```

    }

    linha := make([]any, len(colunas)) // um slice reaproveitado para
    todas as linhas
    _, err := tx.CopyFrom(ctx, pgx.Identifier{"tmp_retorno"}, colunas,
        pgx.CopyFromSlice(len(itens), func(i int) ([]any, error) {
            r := &itens[i]
            linha[0], linha[1], linha[2] = lote.ArquivoID, r.NumeroLinha,
            r.NossoNumero
            linha[3], linha[4], linha[5] = int16(r.CodigoOcorrencia),
            r.DataOcorrencia, r.ValorPagoCentavos
            return linha, nil
        }))
    if err != nil {
        return fmt.Errorf("copy do lote %d: %w", lote.Numero, err)
    }

    if _, err := tx.Exec(ctx, consolidar); err != nil {
        return fmt.Errorf("consolidar lote %d: %w", lote.Numero, err)
    }
    return nil // BeginFunc faz o commit; qualquer erro acima faz
    rollback
})
}

```

Por que esse desenho, ponto a ponto:

- **Atomicidade por lote.** Cópia, consolidação e outbox estão na mesma transação. `pgx.BeginFunc` faz commit se a função devolve `nil` e rollback em qualquer erro, sem a chance de esquecer um `defer tx.Rollback`.
- **Idempotência automática.** Regravar o lote (retry, reprocesso) não muda nada: o `ON CONFLICT DO NOTHING` ignora as linhas existentes, e o `RETURNING` só devolve as inseridas agora, então a outbox não recebe eventos duplicados.
- **Tabela temporária por conexão.** `CREATE TEMP TABLE IF NOT EXISTS ... ON COMMIT DELETE ROWS` cria a tabela uma vez por conexão do pool e só a esvazia a cada commit. É a versão refinada do curso de .NET, que evita criar e destruir objetos de catálogo a cada lote. Tabelas temporárias não geram WAL nem disputam com outras sessões.
- **O slice `linha` reaproveitado.** O `pgx` codifica cada linha no buffer do `COPY` antes de pedir a próxima, então o mesmo `[]any` serve para todas. O que ainda aloca é colocar cada valor num `any` (cerca de um objeto pequeno por coluna). É o custo da API genérica e, com o banco como gargalo, raramente aparece no perfil; se aparecer, implemente a interface `pgx.CopyFromSource` com seus próprios métodos `Next`, `Values` e `Err`.

O `uuid.UUID` da biblioteca padrão é um `[16]byte`; confira na sua versão do `pgx` que ele é codificado direto para `uuid` (como discutido no curso de APIs em Go) e, se não for, converta para `pgtype.UUID` na montagem da linha.

6.4 ON CONFLICT ou MERGE

A regra do curso de .NET vale igual: `ON CONFLICT DO NOTHING` para dados imutáveis, `DO UPDATE ... WHERE` para estado que muda, deduplicação com `DISTINCT ON` na staging quando o mesmo lote pode ter duas linhas para a mesma chave, e `MERGE` (com `RETURNING` desde o PostgreSQL 17) só quando a lógica for rica e as chaves de transações concorrentes não se sobrepuserem, porque ele não tem a garantia atômica de "inserir ou atualizar" do `ON CONFLICT` sob concorrência.

6.5 Alternativas sem COPY: unnest e Batch

Quando o `COPY` não é viável (um pooler como o `PgBouncer` em modo transação com restrições, ou lotes pequenos), passe colunas como arrays. O `pgx` converte slices Go para arrays do PostgreSQL:

```
_, err := tx.Exec(ctx, `
    INSERT INTO retorno_detalhe (arquivo_id, numero_linha, nosso_numero,
    codigo_ocorrencia, data_ocorrencia, valor_pago_centavos)
    SELECT $1, * FROM unnest($2::int[], $3::bigint[], $4::smallint[],
    $5::date[], $6::bigint[])
    ON CONFLICT (arquivo_id, numero_linha) DO NOTHING`,
    lote.ArquivoID, linhas, nossos, ocorrencias, datas, valores) // cada um
um slice Go do mesmo tamanho
```

E para muitos comandos diferentes numa ida só (atualizar o status do arquivo, gravar um checkpoint e inserir um evento), `pgx.Batch` enfileira os comandos e os envia juntos, com os resultados lidos em ordem depois.

6.6 sqlc e o COPY

Se o projeto já usa `sqlc` (como no curso de APIs em Go), ele gera também o código de `COPY`: uma consulta anotada com `:copyfrom` vira um método tipado que recebe um slice de structs de parâmetros e chama o `CopyFrom` do `pgx` por baixo. É mais seguro que montar `[]any` à mão (a ordem e o tipo das colunas são verificados contra o schema) e vale a pena quando as colunas mudam com frequência.

6.7 Ajustes do lado do banco

Os mesmos do curso de .NET, resumidos:

- **Lotes de 1.000 a 10.000 linhas**, medidos. Lotes maiores seguram locks e memória por mais tempo e encarecem o retry.
- **Uma transação por lote**, nunca uma por arquivo de milhões de linhas: transações longas impedem o autovacuum de limpar versões antigas em todas as tabelas.
- **synchronous_commit = off** (por transação, com `SET LOCAL`) só se o dado puder ser reconstruído do arquivo original e a reconciliação detectar a lacuna; para dados financeiros, decida com o time de dados.
- **Autovacuum ajustado** nas tabelas grandes de inserção (`autovacuum_vacuum_insert_scale_factor`).
- **Carga inicial massiva**: remover índices secundários, carregar e recriá-los, nunca com o sistema recebendo escrita.

Exercício do Módulo 6

Crie as tabelas num PostgreSQL local e implemente três escritores: `pgx.Batch` com um `INSERT` por linha, `unnest` e `CopyFrom` + consolidação. Grave 1 milhão de registros com cada um, em lotes de 5.000 e com 4 escritores, e anote linhas por segundo. Rode o `CopyFrom` de novo com o mesmo arquivo e confirme que a contagem de `retorno_detalhe` e da `outbox` não mudou. Se o worker em .NET do outro curso estiver pronto, compare as vazões: com o banco como gargalo, elas devem ficar muito próximas.

Módulo 7 — Leitura em massa

Ler tabelas grandes bem significa, em qualquer linguagem: só as colunas necessárias, linhas consumidas em streaming, paginação por chave em vez de `OFFSET` e um índice que transforme a consulta numa varredura curta. Em Go há duas ferramentas a mais que deixam isso elegante: os **iteradores** da linguagem (`iter.Seq2`, desde o Go 1.23) e o `CopyTo` do `pgx`, que exporta milhões de linhas sem decodificar nenhuma.

7.1 Streaming de linhas

`pool.Query` devolve um cursor: cada `rows.Next()` lê a próxima linha da rede, sem materializar o resultado. Funções como `pgx.CollectRows` montam um slice com tudo e são ótimas para resultados pequenos, nunca para milhões de linhas.

Para oferecer streaming numa API limpa, devolva um iterador. Quem consome usa um `for range` comum, e a consulta é fechada sozinha quando o laço termina ou é interrompido:

```
type Conciliavel struct {
    ID, NossoNumero, ValorPagoCentavos int64
}
```



```

func (r *Repositorio) DoDia(ctx context.Context, dia time.Time)
iter.Seq2[Conciliavel, error] {
    return func(yield func(Conciliavel, error) bool) {
        linhas, err := r.pool.Query(ctx,
            `SELECT id, nosso_numero, valor_pago_centavos FROM
retorno_detalhe WHERE data_ocorrendia = $1`, dia)
        if err != nil {
            yield(Conciliavel{}, err)
            return
        }
        defer linhas.Close()

        var c Conciliavel
        for linhas.Next() {
            if err := linhas.Scan(&c.ID, &c.NossoNumero,
&c.ValorPagoCentavos); err != nil {
                yield(Conciliavel{}, err)
                return
            }
            if !yield(c, nil) {
                return // quem consome parou: o defer fecha a consulta
            }
        }
        if err := linhas.Err(); err != nil {
            yield(Conciliavel{}, err)
        }
    }
}

// uso
for c, err := range repo.DoDia(ctx, dia) {
    if err != nil {
        return err
    }
    // processa uma linha por vez, memória constante
}

```

É o equivalente direto de `AsAsyncEnumerable()` do EF Core, sem o `async`. O mesmo aviso vale: enquanto o laço roda, uma conexão e um snapshot do MVCC ficam presos. Uma leitura em streaming de 40 minutos impede o autovacuum de limpar versões antigas no banco inteiro. Para jobs longos, prefira páginas curtas.

7.2 Keyset em vez de OFFSET

`OFFSET 1000000 LIMIT 1000` lê e descarta um milhão de linhas antes de devolver mil, e cada página fica mais lenta que a anterior. Keyset parte da última chave vista, e a milésima página custa o mesmo que a primeira. Com sqlc:

```
-- name: PaginaDoDia :many
SELECT id, nosso_numero, valor_pago_centavos
FROM retorno_detalhe
WHERE data_ocorrendia = @dia AND id > @depois_de
ORDER BY id
LIMIT @limite;
```

```
var ultimo int64
for {
    pagina, err := q.PaginaDoDia(ctx, banco.PaginaDoDiaParams{Dia: dia,
DepoisDe: ultimo, Limite: 5000})
    if err != nil {
        return err
    }
    if len(pagina) == 0 {
        break
    }
    if err := processar(ctx, pagina); err != nil {
        return err
    }
    ultimo = pagina[len(pagina)-1].ID // a próxima página começa depois da
última chave
}
```

Com um índice em `(data_ocorrendia, id)`, cada página é uma descida na árvore B mais uma leitura sequencial curta. Para ordenações por mais de uma coluna, use a comparação de tuplas do PostgreSQL: `WHERE (data_ocorrendia, id) > (@data, @id)`. E keyset paraleliza bem: divida o intervalo de IDs em faixas e dê uma faixa a cada goroutine de um `errgroup` com `SetLimit`.

7.3 Exportações com COPY TO

Para exportar milhões de linhas em CSV, não é preciso ler linha por linha em Go: o `COPY ... TO STDOUT` faz o PostgreSQL gerar o CSV, e o `pgx` copia os bytes direto para qualquer `io.Writer` (um arquivo, um upload para o S3, a resposta HTTP):

```

conn, err := pool.Acquire(ctx)
if err != nil {
    return err
}
defer conn.Release()

sql := `COPY (SELECT nosso_numero, codigo_ocorrencia, valor_pago_centavos
              FROM retorno_detalhe
              WHERE data_ocorrencia = '` + dia.Format(time.DateOnly) + `')
              TO STDOUT (FORMAT csv, HEADER, DELIMITER ';')`
_, err = conn.Conn().PgConn().CopyTo(ctx, destino, sql)

```

Nenhuma linha é decodificada em structs Go, então o custo de CPU do worker fica perto de zero. O `COPY` não aceita parâmetros: só monte o comando a partir de valores tipados e formatados por você (uma data formatada com `time.DateOnly` é segura; texto vindo de fora nunca é). Se precisar de filtros variáveis, crie uma função no banco e chame-a dentro do `COPY`.

7.4 Escolhendo a ferramenta de acesso

Ferramenta	Melhor para	Custo	Observações
pgx direto	Caminho quente, <code>COPY</code> , consultas dinâmicas	Mínimo	Mais código de <code>Scan</code>
sqlc	Consultas fixas e tipadas	Mínimo (é pgx gerado)	SQL validado contra o schema no build
ORMs (GORM, ent)	Modelos com muitas relações e telas CRUD	Maior	Reflexão e SQL gerado que você não vê; raramente usados em workers de volume

Para este worker, `sqlc` para as consultas de metadados (tabela `arquivo`, `lease`) e `pgx` direto para `CopyFrom`, `CopyTo` e a consolidação é a combinação que dá segurança de tipos sem esconder o SQL.

7.5 Índices

As regras são do PostgreSQL e foram detalhadas no curso de .NET (Módulo 7.5): em índices compostos, igualdade primeiro e intervalo depois; `INCLUDE` para permitir *index-only scans*; índices parciais para subconjuntos pequenos e muito consultados (como a

outbox pendente); BRIN para colunas que crescem junto com a ordem física da tabela; e todo índice cobra seu preço em cada inserção. Confira o plano real com `EXPLAIN (ANALYZE, BUFFERS)`.

7.6 Particionamento

Com dezenas de GB por dia, a tabela de detalhes é particionada por intervalo de data, exatamente como no curso de .NET:

```
CREATE TABLE retorno_detalhe (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  data_ocorrencia date NOT NULL,
  arquivo_id  uuid  NOT NULL,
  numero_linha int  NOT NULL,
  nosso_numero bigint NOT NULL,
  valor_pago_centavos bigint NOT NULL,
  PRIMARY KEY (id, data_ocorrencia),
  UNIQUE (arquivo_id, numero_linha, data_ocorrencia)
) PARTITION BY RANGE (data_ocorrencia);

CREATE TABLE retorno_detalhe_2026_10 PARTITION OF retorno_detalhe
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

Ganha-se *partition pruning*, retenção instantânea (`DETACH + DROP` em vez de `DELETE`) e manutenção por partição. O preço: toda restrição única inclui a coluna de partição, então o `ON CONFLICT` do Módulo 6 passa a ser (`arquivo_id, numero_linha, data_ocorrencia`). Crie partições futuras com antecedência (um job agendado ou a extensão `pg_partman`).

7.7 Prazos e réplicas

Configure um `statement_timeout` para as conexões do worker pelos parâmetros de execução do pgx (`cfgPool.ConnConfig.RuntimeParams["statement_timeout"] = "60s"`), além dos contextos com prazo no código: o primeiro protege o banco mesmo de um bug que esqueça o contexto. Leituras analíticas pesadas (conciliações, relatórios) podem ir para uma réplica com um segundo pool, lembrando do atraso de replicação: o que precisa ler o que acabou de escrever (o fan-in dos chunks) lê do primário.

Exercício do Módulo 7

Com alguns milhões de linhas carregadas pelo Módulo 6, compare a página 1 e a página 500 com `OFFSET` e com keyset usando `EXPLAIN (ANALYZE, BUFFERS)`. Implemente `DoDia` como iterador e confirme que interromper o `for range` no meio (com `break`)

fecha a consulta (a conexão volta ao pool). Por fim, exporte um dia inteiro com `CopyTo` para um arquivo e compare o tempo e a CPU do processo com a mesma exportação feita linha a linha com `Scan` e `encoding/csv`.

Módulo 8 — Confiabilidade: quando algo quebra no meio

Um pipeline confiável é aquele em que qualquer passo pode ser repetido com segurança: entrega pelo menos uma vez (fila, retries, lease) mais idempotência em cada camada (chaves naturais, `ON CONFLICT`, outbox). O desenho é o mesmo do curso de .NET; em Go, três peças ficam particularmente limpas: o lease com `context.WithCancelCause`, o retry escrito em vinte linhas e o relay da outbox como uma goroutine.

8.1 O mapa de falhas

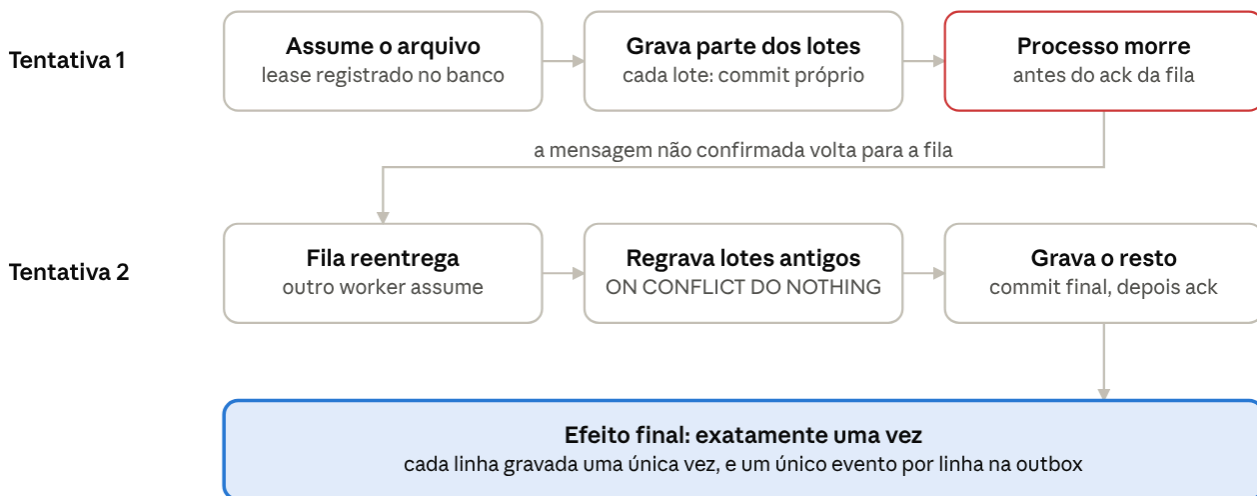
Falha	Exemplo	Resposta do design
Mesmo arquivo enviado duas vezes	Banco reenvia o retorno do dia	<code>UNIQUE (cliente_id, sha256)</code> na ingestão
Worker morre no meio	OOM, deploy, nó removido	Fila reentrega, lease expira, escrita idempotente
Erro transitório	Timeout, failover, deadlock	Retry com backoff e jitter
Erro permanente	Layout inválido	Sem retry: <code>REJEITADO</code>
Gravou, não publicou	Broker fora após o commit	Outbox transacional
Evento publicado duas vezes	Relay cai entre publicar e marcar	Consumidor deduplica pelo ID
Dado incompleto silencioso	Bug perde linhas sem erro	Reconciliação com o trailer

8.2 Idempotência camada por camada

Ingestão (hash único por cliente), tomada do arquivo (`UPDATE` condicional), escrita (`ON CONFLICT DO NOTHING` na chave (`arquivo_id`, `numero_linha`), eventos (a outbox só recebe eventos de linhas realmente inseridas) e consumidores (tabela `inbox` com os IDs já processados). O teste de fogo continua sendo: "posso reprocessar este arquivo do zero agora, em produção, sem medo?".

Pelo menos uma vez + idempotência = efeito de exatamente uma vez

Um arquivo processado duas vezes por causa de uma queda · em vermelho, a falha; em destaque, o resultado



um arquivo, duas tentativas · efeito único

A queda no meio da tentativa 1 não exige tratamento especial: a tentativa 2 refaz tudo do início, e o banco descarta em silêncio o que já tinha sido gravado.

8.3 Lease com context.WithCancelCause

O worker assume o arquivo com o mesmo `UPDATE` condicional do curso de .NET:

```

-- name: TentarAssumir :one
UPDATE arquivo
SET status = 'PROCESSANDO', processando_ate = now() + interval '5 minutes',
    worker_id = @worker_id, tentativas = tentativas + 1
WHERE id = @id
    AND (status = 'RECEBIDO' OR (status = 'PROCESSANDO' AND processando_ate <
now()))
RETURNING tentativas;

-- name: RenovarLease :execrows
UPDATE arquivo SET processando_ate = now() + interval '5 minutes'
WHERE id = @id AND worker_id = @worker_id AND status = 'PROCESSANDO';
  
```

E renova o lease numa goroutine que vive exatamente o tempo do processamento. Se a renovação falhar, ela cancela o contexto do pipeline **com uma causa**, e quem chama sabe por que o trabalho parou:

```

var ErrLeasePerdido = errors.New("lease do arquivo perdido")

func (w *Worker) comLease(ctx context.Context, msg Mensagem, trabalho
func(context.Context) error) error {
  
```

```

ctx, cancelar := context.WithCancelCause(ctx)
defer cancelar(nil) // ao terminar, encerra também a goroutine de
renovação

go func() {
    t := time.NewTicker(time.Minute)
    defer t.Stop()
    for {
        select {
        case <-ctx.Done():
            return
        case <-t.C:
            if err := w.Arquivos.RenovarLease(ctx, msg.ArquivoID); err !=
nil { // 0 linhas = ErrLeasePerdido
                cancelar(fmt.Errorf("%w: %w", ErrLeasePerdido, err))
                return
            }
            _ = w.Fila.EstenderVisibilidade(ctx, msg, 5*time.Minute)
        }
    }
}()

err := trabalho(ctx)
if causa := context.Cause(ctx); errors.Is(causa, ErrLeasePerdido) {
    return causa // o motivo real, não um genérico "context canceled"
}
return err
}

```

`context.Cause` devolve o erro passado a `cancelar`, então o laço do worker distingue "perdi o lease, outro worker assumiu, não confirme a mensagem" de "o processo está desligando" e de uma falha comum. A goroutine de renovação nunca vaza: ela termina quando o contexto termina, e o `defer cancelar(nil)` garante isso em todo caminho de saída.

8.4 Retry com backoff e jitter

Go não tem um Polly na biblioteca padrão, e o essencial cabe em duas funções:

```

func ComRetry(ctx context.Context, tentativas int, base time.Duration, f
func(context.Context) error) error {
    var err error
    for i := range tentativas {
        if err = f(ctx); err == nil || !transitorio(err) {
            return err
        }
    }
}

```

```

        espera := rand.N(base << i) // full jitter: aleatório entre 0 e
base·2^i
        select {
        case <-time.After(espera):
        case <-ctx.Done():
            return errors.Join(err, ctx.Err())
        }
    }
    return err
}

func transitorio(err error) bool {
    if pgErr, ok := errors.AsType[*pgconn.PgError](err); ok {
        switch pgErr.Code {
            case "40001", "40P01", "57P01", "53300": // serialização, deadlock,
servidor reiniciando, conexões esgotadas
                return true
            }
        return false // violação de restrição, erro de sintaxe: repetir não
adianta
    }
    if pgconn.Timeout(err) || pgconn.SafeToRetry(err) {
        return true
    }
    netErr, ok := errors.AsType[net.Error](err)
    return ok && netErr.Timeout()
}

```

E um *decorator* aplica isso ao escritor sem tocar no pipeline:

```

type EscritorComRetry struct{ Interno pipeline.Escritor }

func (e EscritorComRetry) Gravar(ctx context.Context, lote leitor.Lote) error {
    return ComRetry(ctx, 5, 200*time.Millisecond, func(ctx context.Context)
error {
        return e.Interno.Gravar(ctx, lote)
    })
}

```

As mesmas razões do curso de .NET: o backoff exponencial dá tempo para um failover terminar; o **jitter** (`rand.N` de `math/rand/v2` , que é genérico e aceita `time.Duration`) impede que dezenas de réplicas tentem de novo no mesmo instante; só erros transitórios são repetidos; e o retry só é seguro porque `Gravar` é uma transação idempotente. Esse

último ponto cobre até o caso mais traiçoeiro: a conexão cai durante o commit e não se sabe se ele aconteceu. Com idempotência, tanto faz: a nova tentativa grava o que faltar e ignora o resto.

8.5 DLQ e confirmação

Configure a fila para mover mensagens para a DLQ após N recebimentos e use o contador `tentativas` da tabela `arquivo` como segunda barreira. A regra de ouro é a mesma: **apague a mensagem (ack) só depois do commit final** do arquivo. E a DLQ precisa de dono, alarme e uma ação de reenvio.

8.6 O relay da outbox

Uma goroutine em laço, com `FOR UPDATE SKIP LOCKED` para que várias réplicas trabalhem sem pegar os mesmos eventos:

```
type Evento struct {
    ID      uuid.UUID
    Tipo    string
    Chave   string
    Payload []byte // jsonb
}

func (r *Relay) Rodar(ctx context.Context) {
    for {
        n, err := r.publicarLote(ctx)
        if err != nil {
            slog.ErrorContext(ctx, "relay da outbox", "erro", err)
        }
        if n > 0 && err == nil {
            continue // havia eventos: tenta o próximo lote sem esperar
        }
        select {
        case <-ctx.Done():
            return
        case <-time.After(500 * time.Millisecond):
        }
    }
}

func (r *Relay) publicarLote(ctx context.Context) (int, error) {
    var n int
    err := pgx.BeginFunc(ctx, r.pool, func(tx pgx.Tx) error {
        linhas, err := tx.Query(ctx, `SELECT id, tipo, chave, payload FROM
outbox`
```

```

        WHERE publicado_em IS NULL ORDER BY criado_em LIMIT 200 FOR
UPDATE SKIP LOCKED`)
    if err != nil {
        return err
    }
    eventos, err := pgx.CollectRows(linhas, pgx.RowToStructByPos[Evento])
    if err != nil || len(eventos) == 0 {
        return err
    }
    if err := r.publicador.Publicar(ctx, eventos); err != nil { // o ID
vai como ID da mensagem
        return err
    }
    ids := make([]uuid.UUID, len(eventos))
    for i, e := range eventos {
        ids[i] = e.ID
    }
    if _, err := tx.Exec(ctx, `UPDATE outbox SET publicado_em = now()
WHERE id = ANY($1)`, ids); err != nil {
        return err
    }
    n = len(eventos)
    return nil
})
return n, err
}

```

O relay publica **pelo menos uma vez**: se cair depois de publicar e antes do commit, o lote volta a ser publicado. Por isso o ID do evento vai como identificador da mensagem e os consumidores deduplicam. Aqui `pgx.CollectRows` é o certo, porque o lote é pequeno e limitado (200). Os eventos gravados pela outbox são os mesmos que o worker em .NET grava, então um único relay, em qualquer das duas linguagens, atende os dois.

8.7 Reconciliação com o trailer

Os totais do trailer chegam pelo `ContextoDeLeitura` do Módulo 4. A implementação concreta:

```

type ErroDeLinha struct {
    Linha int
    Motivo error
}

type ContextoDoArquivo struct {
    limite          int
    LinhasLidas     int
}

```

```

    LinhasRejeitadas    int
    QuantidadeTrailer   *int    // nil = arquivo sem trailer
    TotalTrailer        *int64
    PrimeirosErros      []ErroDeLinha
}

func NovoContexto(limiteDeErros int) *ContextoDoArquivo { return
&ContextoDoArquivo{limite: limiteDeErros} }

func (c *ContextoDoArquivo) RegistrarErro(n int, motivo error) {
    c.LinhasRejeitadas++
    if len(c.PrimeirosErros) < c.limite {
        c.PrimeirosErros = append(c.PrimeirosErros, ErroDeLinha{n, motivo})
    }
}

func (c *ContextoDoArquivo) RegistrarTrailer(q int, total int64) {
    c.QuantidadeTrailer, c.TotalTrailer = &q, &total }
func (c *ContextoDoArquivo) RegistrarFim(n int) {
    c.LinhasLidas = n }

```

Ele não precisa de mutex porque só a goroutine do leitor escreve nele, e só é lido depois que `g.Wait()` garantiu que ela terminou. As regras são as do curso de .NET:

1. Sem trailer: o arquivo está truncado. `FALHOU`.
2. Quantidade: registros gravados (um `count(*)` por `arquivo_id`) + linhas rejeitadas deve bater com o trailer, ajustado ao manual de cada banco (muitos layouts contam header e trailer).
3. Valor: a soma gravada deve bater com o total do trailer; se houver rejeições, o arquivo não é comparado por valor e vai para revisão, com o relatório de `PrimeirosErros`.

A consulta, a comparação e a mudança de status acontecem numa única transação.

8.8 Desligamento gracioso

No SIGTERM, o contexto do `main` é cancelado. O laço de consumo para de buscar mensagens; os arquivos em andamento têm o contexto cancelado, a transação do lote atual faz rollback e nenhuma mensagem é confirmada. A fila reentrega, o lease expira, outro worker assume e a idempotência cuida das linhas já gravadas. Se preferir terminar o lote em andamento antes de sair, dê aos escritores um contexto derivado de `context.WithoutCancel(ctx)` com prazo curto (alguns segundos): o SIGTERM não interrompe o lote, mas o prazo garante que o processo saia dentro do `terminationGracePeriodSeconds`.

Exercício do Módulo 8

Faça o teste de caos do curso de .NET: processe um arquivo de 2 milhões de linhas com duas réplicas e mate uma delas com `kill -9` no meio, três vezes, em momentos diferentes. Verifique que a contagem de `retorno_detalhe` bate exatamente com o trailer, que não há dois eventos na outbox para a mesma linha e que o arquivo terminou `CONCLUIDO`. Depois derrube o PostgreSQL por 30 segundos durante o processamento e confirme que o `ComRetry` absorve a queda. Se o worker em .NET estiver pronto, rode uma réplica de cada e mate uma: a outra linguagem deve assumir o arquivo sem perceber a diferença.

Módulo 9 — Observabilidade e medição de performance

As perguntas são as do curso de .NET: o pipeline atende a ficha do Módulo 1 em vazão, atraso, erros e saturação? Em Go, as respostas vêm de três fontes: métricas e traces pelo OpenTelemetry, o runtime (que expõe GC, goroutines e memória) e as ferramentas de diagnóstico embutidas na toolchain, `pprof`, `go tool trace` e o flight recorder, que são o equivalente ao conjunto `dotnet-counters`, `dotnet-trace` e `dotnet-dump`.

9.1 O que medir

Métrica	Tipo	Pergunta que responde
<code>retorno.arquivos.processados</code> (por status e banco)	Counter	Quantos terminaram, e como?
<code>retorno.linhas.gravadas</code>	Counter	Qual a vazão real?
<code>retorno.arquivo.duracao</code>	Histogram (s)	Estamos dentro do SLA? Qual o p95 e o p99?
<code>retorno.lote.duracao</code>	Histogram (ms)	O banco está ficando lento?
Idade da mensagem mais antiga na fila	Gauge (s)	Estamos atrasados? Melhor sinal de alerta e de autoscaling
Ocupação do canal de lotes (<code>len(lotes)</code>)	Gauge	Onde está o gargalo dentro do worker?
Espera por conexão do pgxpool	Gauge/Histogram	O pool é pequeno demais ou o banco está saturado?
Goroutines, heap, fração de CPU em GC	Runtime	O processo está saudável?

A ocupação do canal é o melhor diagnóstico interno, como no .NET: canal sempre cheio, o gargalo são os escritores ou o banco; canal sempre vazio, é a leitura. Em Go ela é literalmente `len(lotes)`, lida por um gauge observável.

9.2 Métricas com OpenTelemetry

A configuração dos provedores é a mesma do curso de APIs em Go (Módulo 11.4). No worker, os instrumentos de negócio ficam numa struct criada uma vez:

```
type Metricas struct {
    linhas          metric.Int64Counter
    arquivos        metric.Int64Counter
    duracaoArq      metric.Float64Histogram
    duracaoLote     metric.Float64Histogram
}

func NovasMetricas(pool *pgxpool.Pool) (*Metricas, error) {
    m := otel.Meter("cobranca.retorno")
    var me Metricas
    var err error
    if me.linhas, err = m.Int64Counter("retorno.linhas.gravadas",
metric.WithUnit("{linha}")); err != nil {
        return nil, err
    }
    if me.arquivos, err = m.Int64Counter("retorno.arquivos.processados",
metric.WithUnit("{arquivo}")); err != nil {
        return nil, err
    }
    if me.duracaoArq, err = m.Float64Histogram("retorno.arquivo.duracao",
metric.WithUnit("s")); err != nil {
        return nil, err
    }
    if me.duracaoLote, err = m.Float64Histogram("retorno.lote.duracao",
metric.WithUnit("ms")); err != nil {
        return nil, err
    }

    // Estatísticas do pool lidas a cada coleta, sem custo no caminho quente
    _, err = m.Int64ObservableGauge("retorno.banco.conexoes_em_uso",
        metric.WithInt64Callback(func(_ context.Context, o
metric.Int64Observer) error {
            o.Observe(int64(pool.Stat().AcquiredConns()))
            return nil
        })))
    return &me, err
}
```

```
func (m *Metricas) LoteGravado(ctx context.Context, linhas int, d
time.Duration) {
    m.linhas.Add(ctx, int64(linhas))
    m.duracaoLote.Record(ctx, float64(d.Milliseconds()))
}
```

A mesma regra de cardinalidade do curso de .NET: atributos como `status` e `banco` têm poucos valores; `arquivo_id` e `nosso_numero` nunca são atributos de métrica. O pacote de instrumentação de runtime do OpenTelemetry acrescenta memória, GC e goroutines, e o `pool.Stat()` do pgx expõe também o tempo acumulado de espera por conexão (`AcquireDuration`), que é o sinal mais direto de que o pool ou o banco saturou.

9.3 Traces: a história de um arquivo

Um span por arquivo e um por lote (com amostragem), nunca um por linha. Para que o trace continue o da ingestão, o contexto viaja nos atributos da mensagem: a ingestão injeta o `traceparent` com o propagador do OpenTelemetry, e o worker o extrai antes de abrir o span:

```
ctx = otel.GetTextMapPropagator().Extract(ctx,
portadorDeAtributos(msg.Atributos)) // adapta os atributos da fila
ctx, span := otel.Tracer("cobranca.retorno").Start(ctx, "processar-arquivo",
    trace.WithSpanKind(trace.SpanKindConsumer),
    trace.WithAttributes(attribute.String("arquivo.id",
msg.ArquivoID.String())))
defer span.End()
```

`portadorDeAtributos` é um tipo de poucas linhas que implementa a interface `propagation.TextMapCarrier` (`Get`, `Set`, `Keys`) sobre o mapa de atributos da mensagem. Com o tracer do pgx configurado no pool, cada `CopyFrom` e cada consolidação aparecem como spans filhos, e a linha do tempo vai do upload na API até o último evento da outbox, passando pelas duas linguagens se o sistema misturar workers Go e .NET.

9.4 Logs

As regras do curso de .NET, com `slog`: nunca um log por linha (agregue erros de linha no relatório do arquivo); logs por arquivo e por lote; sempre as variantes com contexto (`InfoContext`, `ErrorContext`) e um handler que acrescenta `trace_id` e `span_id`, como no curso de APIs em Go (Módulo 11.3). No caminho do lote, prefira `log.LogAttrs(ctx, slog.LevelDebug, ...)` com atributos tipados (`slog.Int`, `slog.String`): ele evita alocações quando o nível está desligado.

9.5 Ferramentas de diagnóstico

Ferramenta	Pergunta que responde	Uso típico
<code>pprof CPU</code>	Onde o tempo de CPU está sendo gasto?	30 s sob carga, flame graph
<code>pprof heap</code> e <code>allocs</code>	O que ocupa e o que mais aloca memória?	Crescimento de memória, GC alto
<code>pprof goroutine</code> e <code>goroutineleak</code>	O que cada goroutine faz? Alguma presa para sempre?	Worker travado, vazamento
<code>pprof block</code> e <code>mutex</code>	Onde as goroutines esperam?	Contenção em canais e locks
<code>go tool trace</code>	Como as goroutines, o GC e o escalonador se comportaram no tempo?	Latência irregular, pausas
<code>GODEBUG=gctrace=1</code>	Com que frequência o GC roda e quanto custa?	Ajuste de <code>GOGC</code> e <code>GOMEMLIMIT</code>

O `pprof` fica numa porta de administração só acessível de dentro do cluster (`127.0.0.1:6060`), nunca exposta.

Para problemas intermitentes ("um lote em mil demora 30 segundos"), o **flight recorder** do `runtime/trace` (Go 1.25) é a ferramenta ideal: ele mantém em memória os últimos segundos de trace de execução, com custo baixo, e você grava esse trecho só quando o problema acontece:

```
fr := trace.NewFlightRecorder(trace.FlightRecorderConfig{MinAge: 15 *
time.Second, MaxBytes: 32 << 20})
if err := fr.Start(); err != nil {
    return err
}
defer fr.Stop()

// ... no escritor, depois de cada lote:
if d := time.Since(inicio); d > 10*time.Second && fr.Enabled() {
    if f, err := os.Create(fmt.Sprintf("/tmp/lote-lento-%d.trace",
time.Now().Unix())); err == nil {
        _, _ = fr.WriteTo(f) // os últimos 15 s, para abrir com go tool trace
        f.Close()
    }
}
```

Você obtém exatamente o que aconteceu antes do lote lento (uma pausa de GC, uma goroutine bloqueada, uma conexão esperando) sem ter traçado o processo inteiro o tempo todo. Limite com que frequência o arquivo é gravado, para não encher o disco num dia ruim.

9.6 Benchmarks

Para decisões do caminho quente, o pacote `testing` tem benchmarks embutidos. Desde o Go 1.24, o formato recomendado é `b.Loop()`, que cuida do número de iterações e impede o compilador de eliminar o trabalho medido:

```
func BenchmarkParseDetalhe(b *testing.B) {
    linhas := gerarLinhas(10_000)
    b.ReportAllocs()
    for b.Loop() {
        for i, l := range linhas {
            if _, err := cnab.ParseDetalhe(l, i); err != nil {
                b.Fatal(err)
            }
        }
    }
}
```

Rode com `go test -bench ParseDetalhe -benchmem -count 10 > novo.txt` e compare duas versões com o `benchstat` (de golang.org/x/perf), que calcula a variação e diz se a diferença é estatisticamente significativa. A coluna `allocs/op` é tão importante quanto `ns/op`: em Go, menos alocação significa menos ciclos de GC, que o benchmark isolado não enxerga.

9.7 Teste de carga e alertas

Como no curso de .NET: um ambiente completo (PostgreSQL, um emulador de S3 e de fila), um gerador de arquivos com a distribuição real (muitos pequenos, alguns grandes, um gigante) e carga em degraus até encontrar o ponto em que a vazão para de subir e a latência dispara. Alertas sobre sintomas: arquivo mais antigo na fila há mais da metade do SLA, DLQ com qualquer mensagem, divergência de reconciliação, nenhum arquivo processado numa janela em que costumam chegar muitos. E, específico de Go: número de goroutines crescendo sem parar ao longo de horas.

Exercício do Módulo 9

Instrumente o worker com as métricas de 9.1 e o gauge de `len(lotes)`, suba o Aspire Dashboard como coletor OTLP e rode o teste de carga. Encontre o gargalo só pelas métricas, depois confirme no flame graph da CPU. Ligue o flight recorder, simule um lote

lento (um `pg_sleep` na consolidação) e abra o trace gravado com `go tool trace`. Por fim, compare o `ParseDetalhe` com uma versão que usa `strconv.ParseInt` com `benchstat`.

Módulo 10 — Escala horizontal e deploy

Workers sem estado escalam adicionando réplicas, o sinal certo para escalar é o atraso da fila e o teto é o banco, exatamente como no curso de .NET. O que Go muda no deploy é favorável: um binário estático de poucos MB que sobe em milissegundos, já compilado para código nativo, e um runtime que lê os limites do container sozinho, com uma ressalva importante sobre CPU.

10.1 Escalar pela fila, não pela CPU

Um worker de arquivos passa boa parte do tempo esperando o banco, então a CPU fica baixa enquanto a fila cresce. O autoscaler por CPU conclui que está tudo bem enquanto o SLA estoura. O sinal correto é o trabalho pendente, com o KEDA lendo a fila:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: retorno-worker-go
spec:
  scaleTargetRef:
    name: retorno-worker-go
  minReplicaCount: 1
  maxReplicaCount: 8           # teto calculado pelo banco (10.2)
  pollingInterval: 15
  cooldownPeriod: 300
  triggers:
    - type: aws-sqs-queue
      authenticationRef:
        name: keda-aws-credenciais
      metadata:
        queueURL: https://sqs.sa-east-1.amazonaws.com/<conta>/retornos
        awsRegion: sa-east-1
        queueLength: "4"       # alvo: 4 mensagens por réplica
```

As mesmas ressalvas do curso de .NET: arquivos de tamanhos muito diferentes enganam a contagem de mensagens (uma métrica de linhas pendentes ou da idade da mensagem mais antiga é mais precisa), e escalar a zero custa o tempo de subir um pod no primeiro arquivo do dia, que em Go é dominado pelo agendamento do pod, não pela aplicação.

10.2 O teto é o banco

$$R_{\max} = \left\lfloor \frac{C}{A \cdot E + X} \right\rfloor$$

C é o número de conexões que o banco reserva para o pipeline, A o número de arquivos que cada réplica processa em paralelo (o tamanho do semáforo do Módulo 5.4), E o número de escritores por arquivo e X as conexões extras por réplica (lease, metadados, relay). Com C = 80, A = 2, E = 4 e X = 2, o teto é 8 réplicas, que é o `maxReplicaCount`. No Go, o `MaxConns` do `pgxpool` de cada réplica é exatamente $A \times E + X$: assim, nenhuma réplica consegue abrir mais conexões do que o orçamento lhe dá, mesmo com um bug de concorrência.

Se workers em Go e em .NET dividirem a mesma fila durante uma migração, o orçamento C é único e precisa ser dividido entre os dois `ScaledObjects`.

10.3 O Deployment do worker

```
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: worker
          image: <registro>/cobranca-retorno-worker-go:1.0.0
          resources:
            requests: { cpu: "1", memory: 512Mi }
            limits:   { cpu: "2", memory: 512Mi }
          env:
            - name: GOMEMLIMIT
              value: "460MiB"      # cerca de 90% do limite de memória
          livenessProbe:
            httpGet: { path: /vivo, port: 9090 }
            periodSeconds: 30
            failureThreshold: 4
```

Decisões específicas de Go:

- **Memória menor.** Um worker Go com o parser sem alocação e lotes reaproveitados costuma precisar de bem menos memória que o equivalente em .NET, mas meça no teste de carga antes de copiar este número. O `GOMEMLIMIT` acompanha o limite (Módulo 3.2).
- **Defina um limite de CPU, ou `GOMAXPROCS`.** O runtime usa o **limite** de CPU do container para definir o `GOMAXPROCS` (Go 1.25), não o *request*. Sem limite, num nó de 64 núcleos, o worker usaria 64 threads de execução e o GC paralelo tentaria ocupar todos. Esta é

uma diferença em relação ao conselho do curso de .NET de não limitar CPU: em Go, ou você define o limite, ou fixa `GOMAXPROCS` por variável de ambiente.

- **Liveness por progresso.** Uma porta de administração (a mesma do `pprof`, mas com `/vivo` exposto ao kubelet) responde com base num batimento atualizado pelo laço principal:

```
type Batimento struct{ ultimo atomic.Int64 }

func (b *Batimento) Marcar() { b.ultimo.Store(time.Now().UnixNano()) }

func (b *Batimento) ServeHTTP(w http.ResponseWriter, _ *http.Request) {
    if time.Since(time.Unix(0, b.ultimo.Load())) > 3*time.Minute {
        w.WriteHeader(http.StatusServiceUnavailable) // sem progresso: o
        kubelet reinicia o pod
        return
    }
    w.WriteHeader(http.StatusOK)
}
```

Como no .NET: tolerância folgada (um lote grande pode demorar), e nunca dependa do banco na liveness, porque reiniciar o worker não conserta o banco.

10.4 A imagem e o PGO

O Dockerfile é o mesmo do curso de APIs em Go: compilação com `CGO_ENABLED=0` num estágio de build e uma imagem final *distroless static* com usuário não-root. Não há equivalente a ReadyToRun ou Native AOT para decidir, porque Go já compila para código nativo.

O equivalente ao Dynamic PGO do .NET é o **PGO de build** do Go: colete um perfil de CPU de produção (30 segundos sob carga típica, pelo `pprof`), salve-o como `default.pgo` na pasta do pacote `main` e comite. O `go build` passa a usar o perfil automaticamente para decidir inlining e devirtualização nos caminhos mais quentes. Para um worker dominado por parsing, o ganho costuma ser visível no benchmark; renove o perfil de vez em quando, quando o código mudar bastante.

10.5 Particionando o trabalho

As mesmas estratégias do curso de .NET: fila rápida e fila pesada roteadas pela ingestão conforme o tamanho do arquivo, filas por origem (um banco com arquivos ruins só enche a própria DLQ) e prioridades de negócio, cada partição com seu `ScaledObject` e sua fatia do orçamento de conexões.

10.6 Recuperação de desastre e custo

Como todo processamento pode ser feito a partir do storage: bucket replicado entre regiões, backup do banco com recuperação para um ponto no tempo e um procedimento testado de reenfileirar os arquivos de um período. Nos custos, os mesmos três vilões silenciosos: arquivos originais no storage (regras de ciclo de vida, respeitando a retenção regulatória), volume de logs e armazenamento do banco (partições e retenção).

Exercício do Módulo 10

Num cluster local com KEDA e LocalStack, faça o deploy do worker com o `ScaledObject` e publique 200 arquivos de uma vez; observe as réplicas subindo e voltando após o `cooldownPeriod`. Rode uma vez sem limite de CPU num nó com muitos núcleos e outra com `limits.cpu: 2`, e compare o `GOMAXPROCS` reportado pelo worker na subida e o uso de memória. Colete um perfil de CPU sob carga, gere o `default.pgo`, recompile e compare a vazão do parser com `benchstat`.

Projeto final — o worker de retorno em Go, passo a passo

O projeto final é o mesmo do curso de .NET: um worker que consome a fila, processa arquivos de retorno de qualquer tamanho com memória constante, grava no PostgreSQL de forma idempotente, publica eventos pela outbox e sobrevive a quedas em qualquer ponto. Ele usa as mesmas tabelas e as mesmas mensagens, então pode rodar ao lado do worker em .NET.

Estrutura do módulo

```

cobranca-retorno-go/
├─ go.mod                                go 1.27; sqlc como ferramenta (go get -tool)
├─ cmd/
│   └─ worker/main.go                 main + run do worker
│       └─ relay/main.go               o relay da outbox, implantado separado
├─ internal/
│   └─ cnab/                           RegistroRetorno, ParseDetalhe, ParseTrailer
(Módulo 3)
│   └─ leitor/                         Ler, Lote, ContextoDeLeitura (Módulo 4)
│   └─ pipeline/                       ProcessarArquivo, pool de lotes (Módulo 5)
│   └─ escrita/                        EscritorPostgres, EscritorComRetry (Módulos
6 e 8)
│   └─ confiabilidade/                 lease, ComRetry, ContextoDoArquivo,
reconciliação (Módulo 8)
│   └─ worker/                         Worker, Consumir, processarMensagem
│   └─ fila/ storage/                  SQS e S3 (e versões em memória para testes)
│   └─ banco/                          queries.sql, sqlc.yaml, código gerado
│       └─ observabilidade/            métricas, traces, slog, flight recorder
(Módulo 9)
├─ tests/                             integração com containers
└─ deploy/                             Dockerfile, docker-compose, k8s, ScaledObject

```

As migrations são as do curso de .NET (tabelas `arquivo`, `retorno_detalhe`, `outbox`). Se os workers .NET já as aplicam, o projeto Go só as lê para o `sqlc generate`: um único dono do schema.

Contratos

```

type Mensagem struct {
    ArquivoID      uuid.UUID
    ChaveStorage   string
    Banco          string
    IdRecebimento string          // o receipt handle do SQS
    Atributos      map[string]string // inclui o traceparent (Módulo 9.3)
}

type Fila interface {
    Mensagens(ctx context.Context) <-chan Mensagem // fecha quando ctx é
cancelado
    Confirmar(ctx context.Context, m Mensagem) error
    EstenderVisibilidade(ctx context.Context, m Mensagem, prazo
time.Duration) error
}

```

```

type Storage interface {
    Abrir(ctx context.Context, chave string) (io.ReadCloser, error)
}

type Arquivos interface {
    TentarAssumir(ctx context.Context, id uuid.UUID) (tentativa int, err
error) // ErrJaAssumido se não conseguiu
    RenovarLease(ctx context.Context, id uuid.UUID) error
// ErrLeasePerdido se perdeu
    ReconciliarEConcluir(ctx context.Context, id uuid.UUID, c
*ContextoDoArquivo) (status string, err error)
    MarcarRejeitado(ctx context.Context, id uuid.UUID, motivo string) error
    MarcarFalha(ctx context.Context, id uuid.UUID, motivo string) error
}

```

As interfaces moram no pacote `worker`, que é quem as usa, e cada uma tem uma implementação real e uma em memória para testes.

Orun

```

func run(ctx context.Context, getenv func(string) string) error {
    cfg, err := carregarConfig(getenv) // falha na subida se faltar algo
    if err != nil {
        return fmt.Errorf("configuração: %w", err)
    }
    slog.SetDefault(observabilidade.NovoLogger(os.Stdout, cfg.NivelLog)) //
JSON + trace_id

    desligarTelemetria, err := observabilidade.Iniciar(ctx)
    if err != nil {
        return fmt.Errorf("telemetria: %w", err)
    }
    defer desligar(desligarTelemetria) // com prazo de 5 s

    pool, err := banco.NovoPool(ctx, cfg.BancoURL,
        cfg.ArquivosEmParalelo*cfg.Escritores+2) // MaxConns = A·E + X
(Módulo 10.2)
    if err != nil {
        return fmt.Errorf("banco: %w", err)
    }
    defer pool.Close()

    metricas, err := observabilidade.NovasMetricas(pool)
    if err != nil {
        return err
    }
}

```

```

    batimento := &worker.Batimento{}
    go servirAdmin(ctx, batimento) // pprof + /vivo em :9090, só dentro do
cluster

    w := &worker.Worker{
        Fila:      fila.NovaSQS(cfg.FilaURL),
        Arquivos:  confiabilidade.NovoRepositorio(banco.New(pool),
cfg.IdentidadeWorker),
        Storage:   storage.NovoS3(cfg.Bucket),
        Escritor:  escrita.EscritorComRetry{Interno:
escrita.NovoEscritor(pool)}, // Módulos 6 e 8
        Opcoes:    pipeline.Opcoes{CapacidadeLotes: 4, Escritores:
cfg.Escritores},
        Paralelo:  cfg.ArquivosEmParalelo,
        Metricas:  metricas, Batimento: batimento,
    }

    slog.InfoContext(ctx, "worker iniciado", "gomaxprocs",
runtime.GOMAXPROCS(0))
    w.Consumir(ctx) // bloqueia até o SIGTERM e espera os arquivos em
andamento
    return nil
}

```

O registro de `GOMAXPROCS` na subida não é enfeite: é a forma mais rápida de confirmar que o limite de CPU do pod está sendo respeitado (Módulo 10.3).

O processamento de uma mensagem

Cada linha deste método corresponde a uma decisão de algum módulo:

```

const maxTentativas = 5

func (w *Worker) processarMensagem(ctx context.Context, msg Mensagem) {
    w.Batimento.Marcas()
    inicio := time.Now()
    log := slog.With("arquivo_id", msg.ArquivoID)

    tentativa, err := w.Arquivos.TentarAssumir(ctx, msg.ArquivoID) // Módulo
8.3
    switch {
    case errors.Is(err, ErrJaAssumido):
        _ = w.Fila.Confirmar(ctx, msg) // outro worker está com ele, ou já
terminou
        return
    case err != nil:

```

```

        log.ErrorContext(ctx, "não foi possível assumir o arquivo", "erro",
err)
        return // sem ack: a fila reentrega
    case tentativa > maxTentativas:
        _ = w.Arquivos.MarcasFalha(ctx, msg.ArquivoID, "tentativas
esgotadas")
        _ = w.Fila.Confirmar(ctx, msg)
        return
    }

    contexto := confiabilidade.NovoContexto(1000) // Módulo 8.7
    err = w.comLease(ctx, msg, func(ctx context.Context) error {
        origem, err := w.Storage.Abrir(ctx, msg.ChaveStorage) // stream,
Módulo 4.4
        if err != nil {
            return err
        }
        defer origem.Close()
        return pipeline.ProcessarArquivo(ctx, msg.ArquivoID, origem,
contexto, w.Escritor, w.Opcoes) // Módulo 5
    })

    switch {
    case err == nil:
        status, err := w.Arquivos.ReconciliarEConcluir(ctx, msg.ArquivoID,
contexto)
        if err != nil {
            log.ErrorContext(ctx, "reconciliação", "erro", err)
            return
        }
        _ = w.Fila.Confirmar(ctx, msg) // ack só depois do commit final
        w.Metricas.ArquivoFinalizado(ctx, status, msg.Banco,
time.Since(inicio))
        case errors.Is(err, leitor.ErrLayoutInvalido):
            _ = w.Arquivos.MarcasRejeitado(ctx, msg.ArquivoID, err.Error())
            _ = w.Fila.Confirmar(ctx, msg) // erro permanente: repetir não muda o
conteúdo
        case errors.Is(err, ErrLeasePerdido):
            log.WarnContext(ctx, "lease perdido; quem assumiu termina o arquivo")
        case ctx.Err() != nil:
            // desligando: sem ack; a fila reentrega e a idempotência cuida do
resto
        default:
            log.ErrorContext(ctx, "falha ao processar", "erro", err) // sem ack;

```



```

após N recebimentos, DLQ
    }
    w.Batimento.Marcas()
}

```

Repare que não há retry neste nível. Cada responsabilidade está num lugar: retries transitórios no `EscritorComRetry`, reentrega na fila, duplicatas no banco. Este método só decide **quando confirmar a mensagem**. E o caso de borda mais fino também está coberto: se o SIGTERM chegar entre o commit final e o ack, a mensagem volta, o `TentarAssumir` encontra o arquivo `CONCLUIDO`, devolve `ErrJaAssumido` e a nova entrega é só confirmada.

As onze etapas

- ☐ **1. Fundação.** Módulo Go, `docker-compose` com PostgreSQL e LocalStack, migrations, `sqlc generate`, `run` com configuração validada. Critério: o worker sobe, registra `GOMAXPROCS` e desliga limpo no SIGTERM.
- ☐ **2. Ingestão.** Pode ser a API do curso de APIs em Go (Módulo 6.5) ou um utilitário de linha de comando que envia ao S3, calcula o hash e publica. Critério: o mesmo arquivo enviado duas vezes gera um registro e uma mensagem.
- ☐ **3. Parser.** `cnab.ParseDetalhe` e `ParseTrailer` com testes de tabela e fuzzing. Critério: zero alocações por linha no benchmark; casos de linha curta, data inválida, trailer divergente e última linha sem quebra.
- ☐ **4. Leitor e pipeline.** `leitor.Ler` e `ProcessarArquivo` com um escritor falso. Critério: memória estável com escritor lento; erro no lote 10 interrompe tudo e chega ao chamador.
- ☐ **5. Escritor real.** `EscritorPostgres` com `CopyFrom` e consolidação. Critério: 1 milhão de linhas gravadas; regravar não muda contagens nem gera eventos.
- ☐ **6. Lease.** `TentarAssumir`, `comLease` com `WithCancelCause`. Critério: dois workers com a mesma mensagem, só um processa; lease perdido cancela o pipeline com a causa certa.
- ☐ **7. Resiliência.** `ComRetry`, `EscritorComRetry`, DLQ configurada. Critério: banco fora por 30 s não gera DLQ; layout inválido vai para `REJEITADO` na primeira tentativa.
- ☐ **8. Relay da outbox.** `cmd/relay` com `SKIP LOCKED`. Critério: matar o relay no meio de um lote não duplica efeitos num consumidor com `inbox`.
- ☐ **9. Reconciliação.** `ContextoDoArquivo` e `ReconciliarEConcluir`. Critério: um bug proposital que descarta uma linha é detectado.
- ☐ **10. Observabilidade.** Métricas, traces com propagação pela fila, logs com `trace_id`, flight recorder. Critério: gargalo identificado só pelas métricas; um lote lento gera um trace gravável.

- ☐ **11. Carga, caos e deploy.** Teste de carga em degraus, `kill -9` em momentos diferentes, Kubernetes com KEDA, `GOMEMLIMIT` e limite de CPU, `default.pgo`. Critério: atender a ficha do Módulo 1 com margem e sobreviver a um rollout durante o pico.

Extensão opcional, depois da etapa 9: o fan-out em chunks (Módulo 2.4), com `io.NewSectionReader` ou leitura por `Range` no S3.

Como testar o worker

- **Unidade:** testes de tabela e fuzzing do parser, testes do `ContextoDoArquivo` e do `ComRetry` (com `testing/synctest` para o tempo do backoff, como no curso de APIs em Go).
- **Integração:** `TestMain` sobe PostgreSQL e LocalStack com `testcontainers-go`; o teste monta o `Worker` com as implementações reais apontando para os containers, envia um arquivo, publica a mensagem e espera (com prazo) o status `CONCLUIDO`; depois confere `retorno_detalhe` e `outbox`.
- **Idempotência:** o mesmo teste processa o mesmo arquivo duas vezes e verifica que nada mudou.
- **Concorrência:** toda a suíte roda com `go test -race`, e o teste de integração termina consultando o perfil `goroutineleak`, que precisa estar vazio.
- **Carga e caos:** a etapa 11, num job agendado, comparando a vazão com a medição anterior.

Critérios de aceite finais

- ☐ Vazão sustentada $\geq$ meta de pico da ficha, com 2× de margem
- ☐ O maior arquivo previsto termina dentro do SLA
- ☐ Memória de cada réplica estável durante o maior arquivo, abaixo do `GOMEMLIMIT`
- ☐ Zero alocações por linha no parser
- ☐ Reprocessar qualquer arquivo não altera contagens nem gera eventos
- ☐ `kill -9` em qualquer momento não perde nem duplica registros
- ☐ Toda divergência com o trailer gera alerta
- ☐ `go test -race ./...` limpo e perfil `goroutineleak` vazio após o teste de carga
- ☐ Um worker Go e um worker .NET dividindo a mesma fila produzem o mesmo resultado

Referências e próximos passos

Com os quatro cursos, o sistema de cobrança existe inteiro em duas plataformas: a API que recebe e o pipeline que processa, em .NET 10 e em Go 1.27, sobre o mesmo banco, as mesmas mensagens e o mesmo contrato. Comparar as implementações módulo a módulo é o exercício mais valioso que a série oferece: os problemas são idênticos, e as diferenças mostram o que é essencial (o system design) e o que é convenção de cada ecossistema. Um bom próximo passo é justamente medir os dois workers lado a lado no mesmo teste de carga.

Os outros cursos da série

- [Curso: .NET 10 e System Design para processamento de arquivos em alto volume](#): este mesmo curso em .NET, com os diagramas de GC geracional, `Span`, `PipeReader` e `OFFSET x keyset`.
- [Curso: Minimal APIs no ASP.NET Core 10](#): a API que recebe os arquivos, em .NET.
- [Curso: APIs HTTP em Go](#): a mesma API em Go, com os idiomas da linguagem usados aqui.

Documentação por módulo

Módulo	Leitura
3. Runtime	Guia do GC do Go · sync.Pool
4. I/O	bufio · io
5. Concorrência	Pipelines e cancelamento em Go · errgroup
6 e 7. Banco	pgx · sqlc · PostgreSQL: carga em massa · PostgreSQL: particionamento
8. Confiabilidade	context · Padrão outbox transacional
9. Observabilidade	Diagnósticos em Go · runtime/trace · OpenTelemetry para Go
10. Escala	PGO em Go · KEDA: scaler de AWS SQS

Livros

- *Concurrency in Go*, Katherine Cox-Buday: os padrões de pipeline, cancelamento e limitação de concorrência dos Módulos 5 e 8, em profundidade.

- *100 Go Mistakes and How to Avoid Them*, Teiva Harsanyi: muitos dos erros de fatias, canais e goroutines que este curso evita.
- *Designing Data-Intensive Applications*, Martin Kleppmann: a base teórica de filas, idempotência e processamento em lote, comum a toda a série.

Fontes sobre versões

- [Go 1.27 Release Notes](#): lançado em agosto de 2026; o pacote `uuid`, o perfil `goroutineleak` disponível por padrão, a alocação especializada por tamanho para objetos pequenos e o `encoding/json/v2`.
- [Go 1.26 Release Notes](#): o coletor Green Tea como padrão, `errors.AsType`, mais alocações de slices na pilha e a causa do sinal em `signal.NotifyContext`.
- [Documentação do pacote uuid](#): `NewV7` gera UUIDs ordenados por tempo.